AlmaBetter

Generative & Agentic AI FOUNDATIONS

"**Perfect** for beginners!"

★★★★★

Jeevitha DS

Principal Data Science Instructor

"Builds **strong AI foundations**"

★★★★★

Anirudh Upadhyay

Sr. Data Scientist

"Made complex **AI simple**"

★★★★★

Ayush Singh

VP, Credit Strategy & Analytics

About AlmaBetter

AlmaBetter is India's most trusted Ed-tech platform, helping young Indians launch high-growth careers in **Data Science, AI, Data Analytics, Machine Learning, Web Development, Cloud Computing and DevOps.**

With 1,200+ hiring partners like Amazon, Zomato, and Infosys, we're redefining how India learns tech — one success story at a time!

8 LPA	4.8/5	3000+	40 LPA
Average Package	Student Rating	Placed Students	Highest Package


Arvind Sekaran
Groww


Deepti Goel
amazon


Mukund Jha
EY


Shubhankit S.
accenture


Ronak Satone
Uber


Smruti Ranjan
accenture


Garvit Bhardwaj
snapdeal


Aiman Sahay
Western Union


Shiyas Ali T M
navi


Romaly Das
Myntra


Gaurav Bisht
freecharge


Ayesha Firdose
amazon


Deepanshu P
SWIGGY


Shubham M
amazon

Whether you're a working professional, a fresher, from a non-tech background, or have gap years — it doesn't matter.

Book a **FREE 1:1 call with our Career Expert** and learn how you can also transition into a high-paying tech / AI career with confidence!

Talk to Career Expert

Let's continue with the **AI Foundations....**

Table of Contents

Generative & Agentic AI Foundations

What if you could talk to your files, build an assistant that *thinks like you*, or automate tasks with just one line of text? Welcome to the thrilling world of **Generative AI**, where **LLMs** like **Google's Gemini 1.5 Pro & Flash** are rewriting how we code, create, and communicate. In just five fast-paced lessons, you'll go from understanding **AI prompts** to building **intelligent agents** that not only respond, but take real action. Whether you're aiming to build bots, automate workflows, or revolutionize your apps, this is your **AI launchpad**.

Lesson 1: Introduction to GenAI & Prompt Engineering

Unlock the magic of **Generative AI** by mastering the language it speaks best: **prompts**. In this lesson, you'll learn how to **communicate with LLMs** like **Gemini 1.5** using thoughtful prompt design strategies. Understand the art of crafting **zero shot**, **few shot**, and **chain of thought** prompts that can drastically influence the quality, creativity, and depth of responses. You'll also explore how prompt outcomes can be shaped using key model parameters such as **temperature** which controls randomness, **max tokens** which controls length, and **top p** which controls probability spread. This lesson is all about gaining confidence in prompting and turning simple instructions into powerful AI interactions that feel smart, useful, and human-like.

Click here for Lesson 1 Quiz

Lesson 2: Introduction to LLMs & Working with LLM APIs

This lesson is your hands-on entry into working directly with **LLM APIs**. You'll learn how to connect with **Gemini 1.5 Pro** and **Flash** using your **API key** from **Google AI Studio**. Through practical Python code, you'll send your first prompt, receive a response, and understand the structure of an API call. You'll compare **Gemini 1.5 Pro** which is ideal for complex reasoning, analysis, and creative tasks with **Gemini 1.5 Flash** which is lightweight and fast, perfect for real time use cases. Key concepts like **authentication**, **API response handling**, and **parameter tuning** will be covered in detail. By the end, you'll be able to create your own chatbot and confidently work with LLMs in live applications.

Click here for Lesson 2 Quiz

Lesson 3: Introduction to LangChain & RAG

Take your AI from smart to **context aware** using **LangChain** and the powerful concept of **Retrieval Augmented Generation RAG**. In this lesson, you'll learn how to feed real external information to Gemini models instead of relying only on pre trained knowledge. You'll use **LangChain** to connect prompts, tools, and memory together and learn how to load data, split text, generate **embeddings**, and store them in **vector databases** like **FAISS** or **Chroma**. This enables your app to **retrieve relevant context** dynamically and pass it to Gemini, improving accuracy and reducing hallucinations. You'll also explore how **embedding models**, **retrievers**, and **vector similarity search** form the core of modern RAG systems which are perfect for question answering, document search, and smart assistants.

Click here for Lesson 3 Quiz

Lesson 4: MultiPDFChat – Document Interaction Simplified

What if you could **chat with your PDFs** and the AI would understand them just like a human expert? This is exactly what you'll build in this lesson. Using Gemini and RAG principles, you'll create a **MultiPDF chatbot** that reads, processes, and intelligently answers questions from multiple documents. You'll walk through the process of **loading PDFs**, **splitting text into chunks**, generating **semantic embeddings**, and storing them for **fast retrieval**. Then, when a user asks a question, the system fetches relevant content and sends it to Gemini for a grounded and accurate response. This lesson is ideal for use cases like **academic research**, **legal document review**, **manuals**, or **enterprise knowledge bases**, giving users a fast and natural way to explore large sets of text.

Click here for Lesson 4 Quiz

Lesson 5: Introduction to Agentic AI with n8n

This final lesson moves beyond simple chats and into the future of **Agentic AI** where your model can **think and act**. Using **n8n**, a no code or low code automation tool, you'll design AI powered workflows where Gemini interacts with external apps and services. From sending emails, updating spreadsheets, making API calls, or automating support tickets, your AI becomes a **digital worker**. You'll learn how to set up triggers, connect services like **Google Sheets**, **Slack**, or **Notion**, and use Gemini to write or process the content dynamically. This lesson gives you the power to build **autonomous AI agents** which are systems that understand input, make decisions, and take action — opening the door to real world productivity and automation.

Click here for Lesson 5 Quiz

This chapter lays the **core foundation** for everything that follows. You'll discover what makes **Generative AI** different is it doesn't just analyze, it **creates**. Learn how **LLMs** like **Gemini** generate text, solve problems, and even write code using just a prompt. Then, step into the next evolution: **Agentic AI** where models don't just talk; they **act**, **decide**, and **automate**. This chapter helps you clearly understand the **shift from passive AI to proactive AI**, preparing you to design systems that think *and* do.

Click here for Overall Quiz

Lesson 1: Introduction to GenAI & Prompt Engineering

What is Generative AI?

Imagine this.

You sit at your **computer and type**:
👉 *"Write me a **poem in Shakespeare's style** about **my dog who loves pizza**."*

Within seconds, **your screen lights up** with a beautifully crafted sonnet.
The dog is noble, the pizza divine, and the rhythm feels like it came straight from the Bard himself.

Sounds like science fiction?
Not anymore.
This is **Generative AI** — artificial intelligence that doesn't just analyze the world but *creates something new*.

Breaking It Down: The Simple Definition

Generative AI is a branch of artificial intelligence that can **generate new content**.
Unlike traditional AI systems that classify, detect, or predict, Generative AI can:

- Write **essays, poems, or stories.**
- Create **images, paintings, or product** designs.
- Compose **music or mimic** voices.
- Generate **computer code** or even videos.

In short: **It's AI that creates.**

Why It Feels Magical

For decades, **computers were rule-following machines.**
They only did what **humans programmed** them to do.

Generative AI flips that script.
It learns patterns from **massive datasets** — billions of words, images, and sounds — and uses those patterns to create something original.

Think of it like this:

- **Traditional AI** → "Is this email spam or not?"

- **Generative AI** → “Write me a new email that sounds like it came from a friendly colleague.”

The Shakespeare Analogy

When **Shakespeare** wrote plays, he pulled from his **knowledge of language**, history, and **human emotion.**
Generative AI does something similar: it pulls from **what it has learned in its training** and stitches together content that feels new, yet familiar.

Just as **Shakespeare might reimagine** an old tale in his **unique voice**, AI reimagines data in a creative output.

Why You Should Care

Generative AI isn’t just about **fun poems** or quirky pizza-loving dogs.
It’s reshaping industries:

- **Writers & Marketers** use it to draft ideas.
- **Designers** use it to brainstorm visual concepts.
- **Students** use it to simplify complex topics.
- **Businesses** use it to save time and personalize customer experiences.

This technology is no longer a futuristic dream.
It’s here, and it’s powerful.

How Generative AI is Different from Traditional AI

Artificial Intelligence isn’t new.
For years, we’ve been using AI without even realizing it.

- When your **email filters spam** → that’s AI.
- When **Netflix recommends** your next binge-worthy show → that’s AI.
- When **Google Maps finds** the fastest route home → that’s AI.

But here’s the thing: these are examples of **Traditional AI**.
It’s smart, but it works within limits.

Generative AI, on the other hand, plays in a whole new league.

Traditional AI: The Rule Follower

Traditional AI systems are designed to:

- **Classify** → Is this a cat or a dog?
- **Detection** → Does this X-ray show pneumonia?
- **Predict** → What's the chance of rain tomorrow?

They're incredibly powerful, but they don't *create*.
They follow instructions, like a calculator that always gives you the correct sum.

Generative AI: The Creator

Generative AI is different because it doesn't just *recognize patterns* — it **creates new ones**.

- Instead of telling you if an image is a dog → it can **draw a new dog**.
- Instead of translating text → it can **write a poem in your style**.
- Instead of predicting tomorrow's weather → it can **write a fictional weather report for Mars**.

It's not limited to "yes/no" or "right/wrong."
It opens up the space of **imagination + creativity.**

Analogy: The Student vs. The Artist

- **Traditional AI** is like a top student in math class. Give it equations, and it always solves them correctly.
- **Generative AI** is like an artist. It uses the same knowledge but expresses it in new, surprising, and creative ways.

Both are valuable — but one is predictable, while the other is imaginative.

Why This Difference Matters

This shift from "rule follower" to "creator" is huge.

- In business, it means AI can **design campaigns** instead of just analyzing sales.
- In education, it means AI **can draft essays** instead of just grading them.
- In technology, it means AI can **write code** instead of just running it.

It's like moving from a **calculator… to a co-writer, co-designer, and co-pilot.**

Neural Networks & Transformers

Generative AI is powered by a technology called **neural networks**.
The idea comes from the **human brain: billions of neurons** connected, sending signals.

- **In the brain**: Neurons fire signals to each other, helping us learn and think.
- **In AI**: Artificial "neurons" are arranged in layers, passing numbers instead of signals.

Each layer transforms the input a little, until the network produces an output.

👉 **Example: You give an image of a cat.**

- The first layer detects **lines and edges**.
- Next layers detect **shapes like ears, eyes, whiskers**.
- The final layer says → "It's a cat."

That's the basic neural network idea.

Deep Learning: Going Deeper

When you stack many layers of these neurons, you get a **deep neural network**.
This is why modern AI is often called **deep learning**.

The deeper the network, the more complex the patterns it can recognize:

- **Simple models** → recognize letters.
- **Deep models** → write entire novels.

The Problem: Context

Early neural networks were great at recognizing simple patterns (like cats or dogs).
But when it came to **language**, they struggled.

Example: The word "bank" could mean:

- **A riverbank**
- **A money bank**

Traditional models couldn't handle context well.

The Breakthrough: Transformers

In 2017, researchers introduced the **Transformer architecture**.
This changed everything.

Transformers can:

- Look at **all the words in a sentence at once** (not just one by one).
- Understand context and relationships.
- Scale up to billions of parameters (knowledge points).

This is why Generative AI feels so human-like.

Attention is All You Need

The key idea inside transformers is called **attention**.

It answers the question: *"**When processing this word,** which other words should I pay attention to?"*

Example:
Sentence → ***"The dog chased the ball because it was hungry."***

- Attention helps the AI understand that **"it" = dog**, not ball.

Without **attention,** AI would easily get confused.

Why Transformers Made Generative AI Possible

- They capture **long-term context** → AI remembers what you said earlier.
- They scale to **huge datasets** → trained on billions of books, articles, images.
- They allow **parallel processing** → much faster than old models.

This is why tools like **ChatGPT, Gemini, Claude, and LLaMA** exist today.

Large Language Models

A **Large Language Model (LLM)** is a type of AI model designed to **understand and generate human language**.

- **"Large"** → trained on massive datasets (billions of words).
- **"Language"** → works with human languages (English, Hindi, French… even code).
- **"Model"** → a mathematical system that has learned patterns.

👉 Think of an LLM as a **giant autocomplete on steroids**.
It doesn't just guess the next word — it can write essays, answer questions, and even generate computer programs.

How LLMs Work (Simplified)

1. **Training**:
a. Fed with books, articles, websites, code, and more.
b. Learn statistical patterns: which words often appear together, sentence structures, tone, and style.

2. **Prediction**:
a. When you type a prompt, the model breaks it into tokens (small chunks of words).

b. It predicts the most likely next token, then the next, and so on.

3. **Generation**:
a. Word by word, it builds a coherent sentence, paragraph, or even a novel.

👉 **Example: You type *"Roses are"* →**

- The model predicts "red" as the most likely next word.
- Then continues: "Roses are red, violets are blue…"

Why So Large?

LLMs are called **"large"** because they have:

- **Massive data** → trillions of words from books, Wikipedia, forums, etc.
- **Massive parameters** → billions or even trillions of "knobs" (weights) that store learned knowledge.

More parameters = more knowledge + better understanding of nuance.

Examples of LLMs

- **OpenAI GPT Series** → GPT-3, GPT-4, GPT-4o (ChatGPT).
- **Google Gemini** (formerly Bard).
- **Anthropic Claude**.
- **Meta's LLaMA**.
- **Mistral, Falcon, Cohere Command** → open-source LLMs.

What Can LLMs Do?

- **Answer Questions** → like a tutor.
- **Write Texts** → stories, blogs, code.
- **Summarize** → long reports into bullet points.
- **Translate** → between languages.
- **Reason** (to some extent) → solve puzzles, explain concepts.

Analogy: The Librarian with Super Memory

Imagine a **super-librarian** who has read every book in the world.

- If you ask a question, they won't copy a page word for word.
- Instead, they use what they remember to craft a new answer just for you.

That's how an LLM works — it *remixes knowledge* into new responses.

Limitations of LLMs

- **Not Perfectly Accurate**: Can "hallucinate" wrong facts.
- **Bias**: Reflects biases in the data it was trained on.
- **No True Understanding**: It predicts patterns but doesn't "think" like humans.
- **Costly**: Training requires enormous computing power.

Why LLMs Matter for Generative AI

LLMs are the **engine behind text-based generative AI**.
They make tools like ChatGPT, Gemini, and Claude possible.
They're also spreading into **other areas**: powering code assistants, creative writing tools, educational tutors, and even multimodal systems (text + image + voice).

How Models Learn — Training & Fine-Tuning

From Blank Slate to Wordsmith

When you first open a **newborn baby's mind**, it knows almost nothing.
But as it grows, it learns from experience: **hearing words, observing actions, and connecting patterns.**

Large Language Models (LLMs) go through a similar journey — except their "**childhood**" is compressed into months of intense training on gigantic datasets.

Step 1: Pre-Training (Learning Everything)

- In the beginning, the model is a **blank slate**.
- It is fed massive datasets: books, websites, articles, code, and more.
- The goal: learn **statistical patterns** of language.

👉 **Example:**
It sees **"Once upon a ___" millions of times.**
It learns the blank is usually **"time."**

This pre-training teaches the model **language basics** — grammar, facts, style, relationships between words.

Think of it as giving the model a **world library** to read.

Step 2: Fine-Tuning (Specialization)

Once pre-trained, the model is like a student who has read every book.
But it needs **practice** to be useful in real-world tasks.

Fine-tuning is the stage where developers teach the model:

- How to follow **instructions.**
- How to **stay safe** (avoid harmful outputs).
- How to be **helpful in specific areas** (medical advice, coding, customer service).

👉 **Example:**
A general LLM may know about **medicine.**
Fine-tuning can train it to behave like a polite, careful **medical assistant AI**.

Step 3: RLHF (Reinforcement Learning from Human Feedback)

Even after fine-tuning, the model sometimes gives weird or unsafe answers.
So humans step in:

- They **rank answers** (good, bad, better).
- The model **adjusts to prefer higher-ranked** responses.

This process is called **Reinforcement Learning from Human Feedback (RLHF)**.
It makes the model **aligned** with what humans expect.

👉 **Example:**

- **Without RLHF:** You ask, *"What's 2 + 2?"* It might ramble: *"Mathematics has many branches…"*
- **With RLHF:** It simply answers: *"4."*

Analogy: Training a Chef

- **Pre-training**: The chef reads every cookbook in the world.
- **Fine-tuning**: The chef works in a restaurant, learning to make specific dishes customers like.
- **RLHF**: Diners taste the food and give feedback. The chef improves, adjusting seasoning to please people.

This combination is what makes AI both **knowledgeable** and **useful**.

Why This Matters

Training isn't just about stuffing the model with data.
It's about **shaping** it into something safe, reliable, and aligned with human needs.

That's why your AI assistant today doesn't just blurt out random text — it tries to be helpful, polite, and relevant.

The Power of Prompts

A **prompt** is simply the input you give to a Generative AI system.
It can be:

- **A question** → *"Explain gravity like I'm 10."*
- **An instruction** → *"Write a professional email for a job application."*
- **A style guide** → *"Tell a story in the voice of a pirate."*
- **Even a mix** → *"Create a Shakespearean sonnet about my dog who loves pizza."*

👉 In short: **Prompts are the steering wheel.**
They guide the AI's output, just like asking the right question guides a conversation.

Why Prompts Matter?

AI doesn't "think" like humans. It doesn't know what you mean unless you tell it clearly.
A **well-crafted prompt** = better, more accurate, and more creative results.

Types of Prompts

1. **Zero-Shot Prompting**
a. You ask directly, with no examples.
b. *"Translate this sentence into Spanish."*

2. **Few-Shot Prompting**
a. You provide examples first.
b. *"Here are 3 math problems solved step-by-step. Now solve this one…"*

3. **Role Prompting**
a. You assign the AI a role.
b. *"You are a career coach. Help me prepare for a data analyst interview."*

4. **Chain-of-Thought Prompting**
a. You tell the AI to explain step by step.
b. *"Solve this math problem and show your reasoning."*

5. **Creative Prompting**
a. For fun, imaginative outputs.
b. *"Write a bedtime story about a robot who learns to dream."*

Examples of Prompt Crafting

Weak Prompt:

"Explain AI."

- **Result:** Too broad, generic answer.

Strong Prompt:

"Explain Artificial Intelligence in 3 paragraphs: first in simple terms for a child, second for a college student, and third for a business professional."

- **Result:** Clear, structured, tailored to audiences.

Analogy: Ordering Food at a Restaurant

- If you just say: *"Bring me food."* → The waiter might bring anything.
- If you say: *"Bring me a vegetarian pizza with extra cheese and no onions."* → You'll get exactly what you want.

Prompts are like **food orders.**
The clearer you are, the better the AI can serve you.

Tips for Writing Great Prompts

- Be **specific** → add details about style, format, or audience.
- Providing **context** → background info improves accuracy.
- Break tasks into **steps** → easier for AI to follow.
- Experiment → small tweaks can lead to big differences.

Why Prompting is a Skill

In the world of Generative AI, knowing how to "talk" to AI is a **superpower**.
This skill — called **Prompt Engineering** — is becoming a valuable career path itself.

What Can Generative AI Create?

When you think of creativity, you may imagine artists, musicians, writers, or programmers.
Now, imagine a machine stepping into these roles — not replacing humans, but **collaborating** with us to create.

That's what **Generative AI** does. It doesn't just analyze data — it **produces new content** across multiple formats: text, images, music, video, and even computer code.

1. Text Generation

Text generation is the foundation of Generative AI.

- **How it works**:
  The AI predicts the next word in a sequence, one token at a time, based on patterns it learned from billions of sentences.

- **Capabilities**:

a. Essays, blog posts, and articles.
b. Summaries of long documents.
c. Poetry, stories, and scripts.
d. Emails, speeches, and reports.

- **Real-world uses**:

a. **Businesses**: Drafting marketing copy, newsletters, and reports.
b. **Education**: Creating study notes, simplifying complex topics.
c. **Personal**: Writing wedding vows, bedtime stories, or even entire novels.

👉 **Example Prompt:**
"Write a 500-word blog post explaining climate change in simple terms, with examples suitable for high school students."

2. Image Generation

Generative AI can create visuals that look like they were painted, drawn, or photographed.

- **How it works**:
  Models like *DALL·E*, *Stable Diffusion*, or *MidJourney* are trained on millions of image-text pairs. You describe what you want, and the model paints it pixel by pixel.

- **Capabilities**:

a. Photorealistic pictures.
b. Digital art, illustrations, and concept sketches.
c. Style transfer (e.g., "paint this photo in Van Gogh's style").

- **Real-world uses**:

a. **Marketing & Design**: Quick ad creatives or logo ideas.
b. **Entertainment**: Character designs for games or movies.
c. **Education**: Diagrams, infographics, or science visuals.

👉 **Example Prompt:**
"Create a watercolor-style painting of a cozy cabin in snowy mountains, with smoke rising from the chimney."

3. Music & Audio Generation

Sound is another dimension where AI shines.

1. **How it works**:
   Music AIs are trained on massive collections of melodies, rhythms, and harmonies. Voice models are trained on speech patterns to clone or generate new voices.

2. **Capabilities**:
a. Compose original music in different styles.
b. Generate realistic voices (AI voice cloning).
c. Turn text into speech with human-like intonation.

3. **Real-world uses**:
a. **Film & Gaming**: Background scores generated instantly.
b. **Content Creation**: Podcasters can create intros without hiring a composer.
c. **Accessibility**: Text-to-speech for visually impaired users.

👉 **Example Prompt:**
"Compose a 2-minute lo-fi hip-hop beat with soft piano and background rain sounds."

4. Video Generation

This is the frontier of Generative AI — creating moving pictures.

- **How it works**:
  AI models like *Runway Gen* or *Pika Labs* learn from vast video datasets. They generate motion frame by frame, based on text descriptions or image inputs.

- **Capabilities**:
a. Short clips (seconds to minutes).
b. AI avatars that speak your script.
c. Special effects (turn a sketch into animation).

- **Real-world uses**:
a. **Marketing**: Quick ad prototypes.
b. **Education**: Animated explainers for science lessons.
c. **Film**: Pre-visualization before costly production.

👉 **Example Prompt:**
"Generate a 20-second cinematic video of a spaceship landing on Mars during a dust storm."

5. Code Generation

For programmers, Generative AI is like an intelligent pair programmer.

- **How it works**:
  Models like *GitHub Copilot* or *Code LLaMA* are trained on massive repositories of code. They predict the next line or block of code given a programming context.

- **Capabilities**:

a. Autocomplete functions or full scripts.
b. Debugging and explaining errors.
c. Converting one language into another (Python → Java).

- **Real-world uses**:

a. **Developers**: Faster coding and prototyping.
b. **Beginners**: Learning coding with AI explanations.
c. **Companies**: Automating repetitive coding tasks.

👉 **Example Prompt:**
"Write a Python function that checks if a string is a palindrome."

6. Multimodal Generation: The Next Step

The latest frontier is **multimodal AI** — systems that combine multiple senses (text, image, audio, video).

- **How it works**:
  These models process not just words, but also images, sound, and video, understanding how they connect.

- **Capabilities**:

a. Upload a picture and ask the AI to describe it.
b. Ask AI to generate code after analyzing a diagram.
c. Translate a video into another language with synced audio.

- **Examples**:

a. *GPT-4o (OpenAI)*
b. *Gemini 1.5 (Google)*

- **Real-world uses**:

a. A teacher uploads a math worksheet → AI solves problems step by step.

b. A traveler uploads a menu photo → AI translates dishes instantly.
c. A designer uploads a sketch → AI generates a full-color 3D model.

Analogy: The Infinite Toolbox

Generative AI is like a **limitless creative toolbox**:

- Writers get an endless notebook.
- Artists get an infinite paint palette.
- Musicians get a global orchestra.
- Developers get an always-on coding buddy.

Instead of being bound to one medium, AI can blend them all, opening doors to **collaborative creativity**.

Real-World Applications of Generative AI

Generative AI is not just a **futuristic toy.**
It is already shaping how **businesses operate**, how **students learn**, how **doctors assist** patients, and how we entertain ourselves.

Think of it as **electricity in the early 1900s** — first a novelty, then a necessity.

Let's explore how **Generative AI** is being applied across industries.

1. Business & Marketing

- **Use Cases**:

a. Generate ad copy, social media posts, and product descriptions.
b. Personalize customer emails at scale.
c. Summarize long reports into executive-ready briefs.

- **Example**:
  Coca-Cola used AI tools to create marketing campaigns that combined text, visuals, and music, letting customers even co-create content.

- **Impact**:
  Faster content creation, reduced costs, and more personalized customer experiences.

2. Education & Learning

- **Use Cases**

a. AI tutors explain concepts at different difficulty levels.

b. Generate practice quizzes, lesson summaries, and study guides.
c. Translate materials into multiple languages instantly.

- **Example**:
  Khan Academy introduced *Khanmigo*, an AI-powered tutor that helps students with step-by-step problem solving.

- **Impact**:
  Accessible, customized learning experiences for students of all backgrounds.

3. Healthcare & Medicine

- **Use Cases**:
a. Generate medical notes from doctor-patient conversations.
b. Summarize research papers for faster medical decision-making.
c. Assist in drug discovery by simulating molecular interactions.

- **Example**:
  Some hospitals use AI scribes that listen during consultations and auto-generate clinical notes, saving doctors hours of paperwork.

- **Impact**:
  Doctors spend more time with patients instead of typing. Faster breakthroughs in medical research.

4. Entertainment & Media

- **Use Cases**:
a. Screenwriters draft plots with AI brainstorming tools.
b. Game developers design characters and levels with AI art.
c. Musicians compose songs or experiment with AI collaborators.

- **Example**:
  Netflix has tested AI tools to generate personalized movie trailers tailored to different audiences.

- **Impact**:
  Richer, more personalized content experiences. Creative professionals can prototype ideas faster.

5. Software Development & Technology

- **Use Cases**:

a. Autocomplete code or generate entire functions.
b. Debug errors by explaining code in plain language.
c. Automate repetitive testing tasks.

- **Example**:
  GitHub Copilot, powered by OpenAI's Codex, helps developers write code up to 55% faster.

- **Impact**:
  Speeds up software development, reduces errors, and helps beginners learn coding.

6. Customer Service & Support

- **Use Cases**:
a. AI chatbots handle routine queries 24/7.
b. Summarize customer complaints for human agents.
c. Generate knowledge base articles automatically.

- **Example**:
  Airlines use AI chatbots to answer FAQs and assist with ticket changes.

- **Impact**:
  Lower wait times for customers, reduced burden on support teams.

7. Creative Industries (Art, Fashion, Design)

- **Use Cases**:
a. Generate fashion designs or product prototypes.
b. Convert sketches into finished artworks.
c. Help architects visualize buildings.

- **Example**:
  Fashion brands experiment with AI-generated clothing designs and virtual fashion shows.

- **Impact**:
  Blurs the line between human creativity and machine collaboration.

8. Finance & Business Analytics

- **Use Cases**:
a. Generate financial summaries and reports.
b. Detect fraud by analyzing unusual transaction patterns.

c. Simulate market conditions for risk analysis.

- **Example**:
  Some fintech startups use AI to auto-generate personalized investment advice for clients.

- **Impact**:
  Better insights, faster decisions, improved fraud detection.

Analogy: AI as the "Swiss Army Knife" of Industries

Imagine every industry as a worker with a toolkit.

- Before: Each had hammers, wrenches, and pliers (traditional tools).
- Now: They all share a **Swiss Army Knife** — Generative AI — with blades for writing, drawing, coding, analyzing, and even talking.

It doesn't replace their skills, but **enhances efficiency and creativity**.

Conclusion

Generative AI is moving from curiosity to **core utility**.

- In **business**, it drives efficiency.
- In **education**, it democratizes learning.
- In **healthcare**, it saves time and lives.
- In **entertainment**, it powers creativity.
- In **technology**, it accelerates innovation.

Benefits & Opportunities of Generative AI

AI acts like a **24/7 digital assistant**.

- Summarizes reports
- Drafts emails
- Writes code

👉 **Result:** Hours of work → done in minutes.

🎯 Personalized Experiences

Generative AI adapts to **you**.

- Custom learning paths
- Tailored marketing
- Personalized playlists

👉 **Result:** Feels like the tech truly “knows” you.

🎨 Creativity Unlocked

Stuck with **writer’s block**?
AI sparks fresh ideas:

- Story plots
- Song melodies
- Art concepts

👉 **Result:** AI doesn’t replace creativity — it **ignites** it.

💡 Faster Prototyping, Lower Costs

Startups and individuals can:

- Design logos
- Build websites
- Draft pitch decks

👉 **Result: Big-budget creativity** on a small budget.

🌍 Democratization of Skills

No coding? No design degree? No problem.
AI makes **expertise accessible** to everyone.

👉 **Result:** Anyone can create, innovate, and compete.

📞 Smarter Customer Support

AI chatbots = **instant 24/7 help**.

- Solve FAQs
- Book tickets
- Troubleshoot issues

👉 **Result:** Happy customers + reduced costs.

🧑‍💻 New Career Paths

Generative AI is creating jobs:

- **Prompt Engineers**
- **AI Trainers**
- **AI Content Specialists**

👉 **Result:** Entirely new **industries emerging**.

🤝 Human + AI Collaboration

Humans bring **context, judgment, and empathy**.
AI brings **speed, scale, and ideas**.
Together → unbeatable.

✅ **Key Takeaway:**
Generative AI is not just a tool.
It's an **opportunity engine** → boosting **productivity, creativity, accessibility, and careers**.

Risks & Challenges of Generative AI

Accuracy & Misinformation

AI can sound **confident but wrong**.

- Fake facts
- Misleading content
- "Hallucinations" (made-up answers)

Risk: Trust issues if unchecked.

Bias & Fairness

AI learns from data — and data has **bias**.

- Gender stereotypes
- Cultural bias
- Skewed recommendations

Risk: Reinforces unfair patterns.

Copyright & Ownership

Who owns AI-created work?

- The user?
- The company?
- The AI?

Risk: Legal battles over **intellectual property**.

Privacy Concerns

Feeding sensitive info into AI can be dangerous.

- Personal data leaks
- Corporate secrets exposure

Risk: Data misuse if safeguards fail.

Job Disruption

AI can **automate tasks** once done by humans.

- Writers
- Designers
- Customer service reps

Risk: Job shifts → workers need **reskilling**.

Over-Reliance on AI

If humans depend too much on AI…

- Critical thinking weakens
- Creativity narrows
- Wrong outputs go unquestioned

Risk: Losing **human judgment** in the loop.

Key Takeaway

Generative AI brings **power** — but also **pitfalls**.
The challenge: **maximize benefits while minimizing risks** through responsible use, regulation, and human oversight.

The Future of Generative AI

Smarter AI Models

Models are growing **bigger and faster**.

- More context awareness
- Better reasoning
- Multi-language mastery

Future: AI that feels more like a **thinking partner**.

Multimodal AI

Not just text. AI will **see, hear, and create**.

- Text → Image, Video, Music
- Voice → Text + Actions
- Mixed reality applications

Future: One AI that blends **all senses**.

Personal AI Assistants

Imagine an AI that **knows you**.

- Custom learning tutor
- Personalized job coach
- Health and wellness guide

Future: AI that adapts to **your life**.

Collaboration, Not Replacement

AI won't just replace — it will **enhance**.

- Humans + AI = **co-creators**
- New roles: **AI trainers, explainers, ethicists**

Future: Work shaped by **partnership**, not competition.

Regulation & Ethics

As AI spreads, rules will grow.

- Laws on copyright
- Guidelines for fairness
- Global AI policies

Future: A balance between **innovation and safety**.

Key Takeaway

Generative AI's journey is just beginning.
The future isn't AI **vs** humans.
It's AI **with** humans — building a smarter, fairer, more creative world.

What We Learned

- **What** Generative AI is: Machines creating like humans
- **How** it works: **LLMs, training, prompts**
- **Where** it's used: **Business, art, education, healthcare**
- **Why it matters:** Boosts **creativity, efficiency, access**
- **Risks & challenges: Bias, misinformation, privacy, jobs**
- **Future:** AI as a **partner, not a rival**

The Road Ahead

Generative AI is still **young**.
Its next chapters will be written by:

- Innovators
- Policymakers
- Everyday users like **you**

The question isn't: *"What can AI do?"*
The real question is: **"What do we want AI to do for us?"**

Final Thought

Generative AI is not the end of human creativity.
It's the **beginning of amplified creativity** — where humans and machines imagine, build, and dream together.

Lesson 2 : Introduction to Large Language Models (LLMs)

Imagine this:
You start typing ***"Once upon a…"*** and your computer instantly writes a full fairy tale, complete with heroes, dragons, and a moral ending.

That's the magic of **Large Language Models (LLMs).**

What is a Language Model?

At its core, a **language model** is like a very smart predictor.

- When you text someone and your phone suggests the next word, that's a small language model.
- Now imagine the same idea — but trained on **millions of books, articles, and websites.** Suddenly, the model doesn't just suggest "pizza" after "I want a…" — it can write recipes, business emails, or even Shakespearean poetry about your dog.

What Makes It "Large"?

The **"Large"** in LLM isn't about physical size. It's about two things:

- **Data:** Trained on everything from Wikipedia to Reddit to classic novels.
- **Parameters:** Think of them as "neurons." GPT-4 has **trillions** of them.

Example:

- A small model might know how to complete:
  "The capital of France is …" → ***"Paris."***
- A large model can answer:
  "Explain the cultural impact of the French Revolution in 3 bullet points."

Why Do LLMs Matter?

LLMs are like **Swiss Army knives of language.**

- Want to **summarize** a 50-page report in 5 minutes? Done.
- Need to **translate** English into Japanese slang? Easy.
- Struggling to **debug Python code at 2 AM**? LLMs have your back.

They are the hidden engines behind **ChatGPT, Google Gemini, Claude, and LLaMA.**

💡 **Fun Thought Experiment:**
If Shakespeare had access to an LLM, he might have written:
"Shall I compare thee to a JavaScript array?"

How LLMs Work

Think of an LLM as a **super-charged autocomplete**.
When you type *"I love eating…"* — your phone might guess *"pizza."*
But an LLM can go much further:

"I love eating pizza because the molten cheese stretches like golden threads, and every bite feels like a festival in my mouth."

How does it do that? Let's break it down.

Step 1: Breaking Language into Pieces (Tokens)

LLMs don't "see" words the way we do. They split text into **tiny chunks** called **tokens**.

- Example: ***"Chatbots are cool"*** → might become ["Chat", "bots", "are", "cool"].
- This is like Lego blocks. The model builds sentences by snapping tokens together.

Step 2: Learning Patterns

During training, the model reads billions of sentences and learns patterns.

- If it sees ***"peanut butter and …"*** millions of times, it learns to predict *"jelly."*
- If it sees code like `print("Hello…")`, it learns the next token is usually `"World"`.

It's not memorization — it's pattern recognition at a massive scale.

Step 3: Attention — The Secret Sauce

LLMs use a mechanism called **attention**.
Think of it like a spotlight in your brain: when you read a sentence, you don't focus on every word equally — some words matter more.

Example:
Sentence: *"The dog that chased the cat was hungry."*
Question: Who was hungry?

- Attention helps the model figure out it's **the dog**, not the cat.

Step 4: Generating Text

When you ask a question, the model generates one token at a time.

- **Prompt:** *"Tell me a joke about cats."*
- **Model:** "Why did the cat sit on the computer? …"
- **Next token:** "… because it wanted to keep an eye on the mouse!"

Simple Analogy

An LLM is like a **musician who has listened to every song ever written**.

- It doesn't know music theory the way humans do.
- But if you give it a starting note, it can *improvise* a new melody that sounds real.

Popular LLM Families

LLMs are like **superhero families** — each with their own powers, quirks, and favorite costumes. Let's meet the main ones:

1. OpenAI's GPT (Generative Pre-trained Transformer)

- **Members:** GPT-3, GPT-3.5, GPT-4, GPT-4o (the "multimodal" sibling).
- **Superpower:** Can handle text, code, and with GPT-4o, even images and audio.
- **Example:**
  You: *"Write a poem about coffee in Shakespearean style."*
  GPT: *"Oh bitter brew that warms the weary soul…"*

2. Anthropic's Claude

- **Members:** Claude 1 → 2 → 3 (latest is very good at reasoning).
- **Superpower:** Safer, more aligned, polite — like the "wise monk" of AI.
- **Example:**
  You: *"Summarize a 50-page report in plain English."*
  Claude: *"Here's a 3-paragraph summary with key takeaways in bullet points."*

3. Google DeepMind's Gemini (formerly Bard)

- **Members:** Gemini 1, Gemini 1.5 (multimodal and efficient).
- **Superpower:** Can reason across text, code, and images. Strong with Google ecosystem.
- **Example:**
  You upload a chart. Gemini says: *"This trend shows sales dipping in Q3, likely due to supply chain delays."*

4. Meta's LLaMA (Large Language Model Meta AI)

- **Members:** LLaMA 1, 2, 3 (open-source).
- **Superpower:** Free, customizable — the "DIY" hero.
- **Example:**
  Startups use LLaMA to build chatbots without paying per API call.

5. Mistral

- **Members:** Mistral 7B, Mixtral (a "mixture of experts").
- **Superpower:** Lightweight, fast, efficient. Think of it as the **parkour ninja** of LLMs.
- **Example:**
  Great for edge devices — like running chatbots on a laptop instead of a giant server.

6. Others Worth Knowing

- **Cohere's Command R** → tuned for retrieval-augmented generation (RAG).
- **AI21's Jurassic** → early player in LLM space.
- **xAI's Grok** → Elon Musk's edgy chatbot inside X (Twitter).

Quick Analogy

- GPT = **All-rounder celebrity**
- Claude = **Wise monk**
- Gemini = **Google-backed scientist**
- LLaMA = **Open-source hacker**
- Mistral = **Fast parkour ninja**

💡 **Key Takeaway:**
Not all LLMs are the same. Choosing the right one depends on:

- **Cost** (free vs. paid APIs)
- **Strengths** (reasoning, creativity, speed)
- **Ecosystem** (OpenAI, Google, Meta, Anthropic, etc.)

APIs & Why They Matter

Imagine you're at a **restaurant.**

- You don't walk into the kitchen to cook.
- Instead, you tell the waiter: *"One Margherita pizza, please."*
- The waiter carries your request to the chef, then brings the finished dish to your table.

That waiter? **That's an API.**

What is an API?

- **API = Application Programming Interface**
- A set of rules that lets two systems "talk" without you seeing the messy details.

For LLMs:

- You don't need to know how GPT-4 was trained on trillions of words.
- You just send a request (your "pizza order") → get a response (a story, code, summary, etc.).

Why Do APIs Matter for LLMs?

1. **Accessibility** → You don't need a supercomputer. Just call the API.
2. **Flexibility** → One API → many apps (chatbots, assistants, code copilots).
3. **Scalability** → Build once, serve thousands of users.

Example:

```
import openai
response = openai.ChatCompletion.create(
  model="gpt-4",
  messages=[{"role": "user", "content": "Write a haiku about data"}])
print(response["choices"][0]["message"]["content"])
```

Output:
"Silent streams of bits,
Patterns whispering insights,
Data dreams unfold."

💡 **Key Insight:**
APIs are the **bridge** between you and the giant brain of an LLM. Without APIs, LLMs would remain locked away in research labs.

Accessing Popular LLM APIs

LLMs are like different **clubs in the city** — each with its own entry rules. To get in, you usually need a **membership card (API key).** Once you have it, you can order whatever you want — stories, summaries, or even working code.

Let's explore how to get inside the most popular clubs:

1. OpenAI (GPT-3.5, GPT-4, GPT-4o)

- **Get Access:**

a. Sign up at platform.openai.com.
b. Generate an **API Key** from your account dashboard.

```
from openai import OpenAI
client = OpenAI(api_key="your_api_key")
response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[{"role": "user", "content": "Tell me a fun fact about space"}]
)
print(response.choices[0].message.content)
```

2. Anthropic (Claude 3 Family)

- **Get Access:**

a. Visit console.anthropic.com.
b. Create an **API Key.**

```
import anthropic
client = anthropic.Anthropic(api_key="your_api_key")
response = client.messages.create(
    model="claude-3-opus-20240229",
    messages=[{"role": "user", "content": "Explain AI like I'm 10"}],
    max_tokens=100
)
print(response.content[0].text)
```

3. Google DeepMind (Gemini)

- **Get Access:**

a. Go to ai.google.dev.
b. Enable **Google AI Studio** → create an **API key.**

```
import google.generativeai as genai
genai.configure(api_key="your_api_key")
model = genai.GenerativeModel("gemini-1.5-flash")
response = model.generate_content("Write a short joke about bananas")
print(response.text)
```

4. Meta's LLaMA (via Open Source or Providers like Replicate/Hugging Face)

- **Get Access:**

a. Download weights (requires approval) → Meta's LLaMA
b. Or use the Hugging **Face Inference API**.

```
from huggingface_hub import InferenceClient
client = InferenceClient("meta-llama/Llama-2-7b-chat-hf")
response = client.text_generation("Give me a motivational quote")
print(response)
```

5. Mistral (Mistral 7B, Mixtral)

- **Get Access:**

a. Head to mistral.ai.
b. Request API key.

```
import requests
url = "https://api.mistral.ai/v1/chat/completions"
headers = {"Authorization": "Bearer your_api_key"}
data = {
  "model": "mistral-tiny",
  "messages": [{"role": "user", "content": "Summarize Romeo and Juliet in 3 lines"}]
}
response = requests.post(url, headers=headers, json=data)
print(response.json()["choices"][0]["message"]["content"])
```

Quick Recap

- **API Key** = membership card
- **Request** = placing an order
- **Response** = your dish (story, code, poem, insight)

💡 **Key Takeaway:**
Every major LLM gives you an **API playground.** Once you master one, the others feel familiar — just different flavors of the same dish.

Working with Google Gemini API

Google **Gemini** is a family of **Large Language Models (LLMs)** developed by **Google DeepMind**. It can power **text, code, image, and multimodal tasks**, making it one of the most versatile LLMs available.

This chapter will walk you through:

- Getting started with Gemini API (free)
- Installing the SDK
- Sending your first prompt
- Exploring available models
- Using role-based prompts (customer support & tutoring examples)

1. Quick Setup (Free Access)

- Go to Google AI Studio
- Sign in with your Google account
- Create an API key:-
  a. Click your profile (top-right) → **API Keys**
  b. Click **Create API Key**
  c. Copy and save it (you'll need this in your code)

👉 **Why?** The API key authenticates your program and gives you access to Gemini's capabilities.

2. Install the Gemini SDK

Run in **Colab or your terminal**:

```
pip install google-generativeai
```

👉 **Why?** This installs the official Gemini Python SDK, which makes it easy to interact with the models without writing low-level networking code.

3. List All Available Gemini Models

Once installed, you can see which models are available:

```
import google.generativeai as genai
genai.configure(api_key="GOOGLE_API_KEY")  # Replace with your key
for m in genai.list_models():
```

```
print(m.name)
```

Sample Output (shortened):

models/gemini-1.5-flash
models/gemini-1.5-pro
models/gemini-2.5-flash
models/gemini-2.5-pro
models/embedding-001
models/imagen-4.0-generate-preview

👉 **Why?** Gemini is not one model — it's a **family**. For example:

- gemini-1.5-flash: Fast, cost-efficient (good for chatbots, everyday apps)
- gemini-1.5-pro: More powerful, better reasoning (good for analysis, coding)
- imagen-*: Image generation models
- embedding-*: Embedding models for search & semantic similarity

4. First Prompt with Gemini 1.5 Flash

```
model = genai.GenerativeModel("gemini-1.5-flash")
response = model.generate_content("Tell me a fun fact about space.")
print(response.text)
```

Output:
There are more stars in the universe than grains of sand on all the beaches on Earth.

👉 **Why?** This demonstrates the **basic workflow**: choose a model → send a prompt → receive text output.

5. Prompt-Based Interactions

Example 1: Customer Support Assistant

```
model = genai.GenerativeModel("gemini-1.5-flash")
prompt = "Act as a support agent. A customer says: 'I was charged twice for the same item. What should I do?'"
response = model.generate_content(prompt)
print(response.text)
```

Sample Output:

- Apology for the issue
- Request for details (order number, date, screenshot)
- Assurance of refund

👉 **Why?** By assigning a **role** in your prompt ("Act as a support agent"), you guide Gemini's tone and structure.

Example 2: Math Problem Explainer

```
prompt = "Explain how to solve: 2x + 3 = 11. Break it into clear steps for a 7th-grade student."
response = model.generate_content(prompt)
print(response.text)
```

Sample Output (simplified):

1. Subtract 3 from both sides → 2x = 8
2. Divide both sides by 2 → x = 4
3. Check the answer → 2(4)+3 = 11 ✅

👉 **Why?** The level of explanation changes based on how you **frame the task** ("for a 7th-grade student"). This is the essence of **prompt engineering**.

Hands-On Walkthrough: Building a Chatbot with Google Gemini API

Goal:
Create a simple, intelligent chatbot powered by **Google Gemini 1.5 Flash**, capable of human-like responses — using only Python.

Step 1: Install Required Libraries

We need a few packages to work with Gemini and to build a UI later.

Package	Purpose
python-dotenv	Load your API key from a .env file securely.

pdf2image	Convert PDFs to images (useful if you later want to chat wi PDFs).
Pillow (PIL)	Handle images in Python (screenshots, inputs to Gemini).
google-generativeai	Official Gemini SDK for Python.
gradio	Build a simple, shareable chatbot interface.

Colab / Terminal Command:

```
!pip install python-dotenv pdf2image Pillow google-generativeai gradio
```

Step 2: Load Your API Key Securely

Put your key in a .env file:

```
.env
GOOGLE_API_KEY="your-api-key-here"
```

Python code to load it:

```
from dotenv import load_dotenv
import os
load_dotenv()  # loads all keys from .env
```

Why?
Storing the key in .env avoids accidentally exposing it in your code or GitHub repo.

Step 3: Import Required Libraries

```
import base64
import os
import io
from PIL import Image            # Image handling
import pdf2image                 # Convert PDFs to images
import google.generativeai as genai  # Gemini SDK
```

Step 4: Configure the Gemini API

```
genai.configure(api_key=os.getenv("GOOGLE_API_KEY"))
```

This line:

- Fetches the key from your environment variable.
- Authenticates all Gemini API calls.

Step 5: List Available Models

```
for m in genai.list_models():
  if "generateContent" in m.supported_generation_methods:
    print(m.name)
```

You'll see models like:

- gemini-1.5-flash → **fast, free tier**, great for chatbots.
- gemini-1.5-pro → more powerful reasoning, larger context.

Step 6: Define the Chat Function

```
def chat_with_gemini(user_input):
  model = genai.GenerativeModel('gemini-1.5-flash')
  prompt = f"""You are a Senior Data Scientist at Google.
  Answer user questions with clarity and insight.
  User: {user_input} Bot:"""
  response = model.generate_content([prompt])
  return response.text
```

Explanation:

- genai.GenerativeModel() loads Gemini.
- prompt sets a **persona** (role-based prompting).
- generate_content([prompt]) sends the request and gets a reply.

Step 7: Run a Terminal Chatbot

```
def run_chatbot():
  print("Welcome to the Gemini Chatbot! Type 'exit' to quit.")
  while True:
    user_input = input("You: ")
    if user_input.lower() == 'exit':
      print("Chat ended.")
```

```
        break
    response = chat_with_gemini(user_input)
    print(f"Bot: {response}")
run_chatbot()
```

Here:

- Infinite loop until the user types exit.
- Each input is passed to Gemini → response printed back.

Step 8: Clean Responses (Remove Markdown)

Sometimes Gemini replies with **Markdown** formatting. Let's clean it:

```
def to_markdown(text):
    text = text.replace("## ", "")
    text = text.replace("#", "")
    text = text.replace("**", "")
    text = text.replace("* ", "")
    text = text.replace("•", " *")
    return text
```

Step 9: Enhanced Chat Function

```
def chat_with_gemini(user_input):
    model = genai.GenerativeModel('gemini-1.5-flash')

    prompt = f"""You are a Senior Data Scientist at Google.
    User: {user_input}
    Bot:"""
    response = model.generate_content([prompt])
    cleaned_response = to_markdown(response.text)
    return cleaned_response
```

Now responses look **cleaner and readable**.

Step 10: Run the Enhanced Chatbot

```
run_chatbot()
```

Same loop as before, but responses are Markdown-free.

Step 11: Build a Web UI with Gradio

Gradio turns our chatbot into a simple web app.

```
import gradio as gr
def chatbot_interface(user_input):
    return chat_with_gemini(user_input)
iface = gr.Interface(
    fn=chatbot_interface,
    inputs="text",
    outputs="text",
    title="Gemini Chatbot",
    description="Chatbot powered by Gemini 1.5 Flash. Ask me anything!"
)
iface.launch()
```

- fn=chatbot_interface → connects user input to Gemini.
- inputs="text", outputs="text" → text in, text out.
- launch() → opens in your browser.

✅ Result: A **shareable chatbot UI** anyone can try.

👉 This gives you a **working chatbot in both terminal and browser** using Gemini.

Lesson 3: Introduction to LangChain & RAG

What is LangChain?

LangChain is like a smart toolkit that helps developers build better AI apps using ChatGPT-like models.

What's the problem it solves?

Language models (like ChatGPT) are smart, but:

- They don't always know specific information, like your company's private data or real-time updates.
- They're generalists, not specialists in fields like medicine or law unless you give them extra help.

🔧 What does LangChain do?

LangChain helps you:

- **Connect language models to other tools or data**, like:
  a. 📚 Databases
  b. 🌐 Websites
  c. 📄 Your documents
- **Add memory** so the AI remembers what was said earlier (like a human conversation).
- **Build chains of logic**, so it can perform multiple steps, like:
  search → summarize → answer

🧪 Real-life Example

Say you want to build a **chatbot that answers legal questions** using actual law documents from your firm:

- A plain language model might **guess** answers.
- **LangChain helps** connect the model to your firm's **document database**, so it gives **real, accurate** answers from the right source.

LangChain Components

There are six main areas that LangChain is designed to help with.

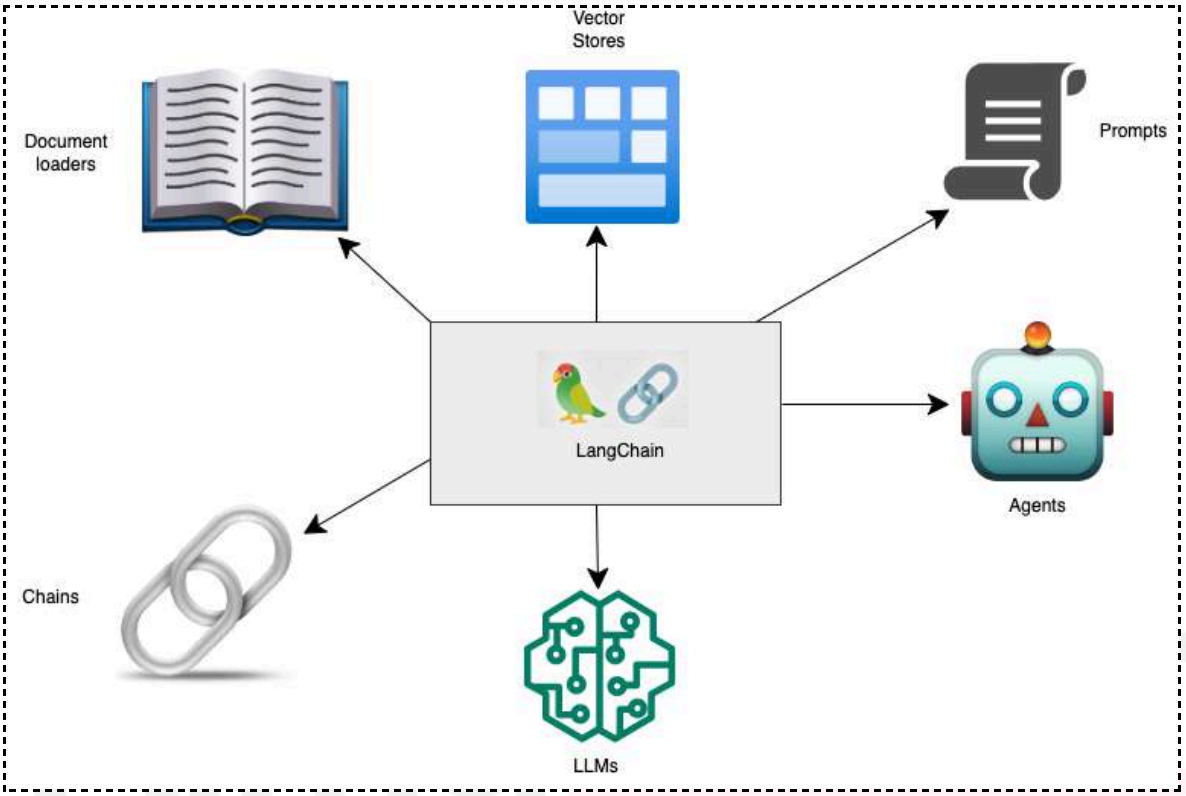
1. Model I/O
2. Data Connection
3. Chains
4. Agents
5. Memory
6. Collbacks

The framework also allows integration with many tools to develop full-stack applications, such as OpenAI, Huggingface Transformers, and Vectors stores like Pinecone and chroma db, among others.

Comprehensive Explanation of Components

- **Model I/O:** Interface with language models. It consists of Prompts, Models, and Output parsers
- **Data connection:** Interface with application-specific data sources with data transformers, text splitters, vector stores, and retrievers
- **Chains:** Construct a sequence of calls with other components of the AI application. Some examples of chains are sequential chains, summarization chain, and Retrieval Q&A Chains
- **Agents:** LangChain introduces the notion of agents, which are entities that can perform specific tasks or actions. Agents can be used to automate interactions with LLMs, such as executing commands or retrieving information based on natural language inputs.

- **Memory:** The framework supports memory management, enabling applications to maintain context across interactions. This is particularly useful for creating conversational agents that need to remember previous interactions.
- **Callbacks:** Log and stream steps of sequential chains in order to run the chains efficiently and monitor the resources consumption



Chains in Langchain

In **LangChain**, **chains** are powerful, reusable components that can be **linked together** to perform complex, multi-step tasks.

Chains allow developers to build **structured workflows** around LLMs by combining multiple components like:

- **Prompt templates**
- **LLMs**
- **Tools**
- **Memory**
- **APIs**
- **Custom logic**

💡 Why Are Chains Important?

Chains are invaluable due to their ability to:

- **Blend diverse components** (e.g., prompts, retrievers, tools)
- **Simplify complex workflows**
- **Promote reusability and modularity**
- **Create context-aware, interactive applications**

By structuring your logic into chains, your application becomes more **readable**, **maintainable**, and **scalable**.

🧠 Example: How a Simple Chain Works

Imagine this flow:

1. A user provides **raw input**
2. The input is **polished** using a PromptTemplate
3. This refined prompt is passed to an **LLM**
4. The **output** is returned or passed to the next step

This chain simplifies the overall process while enriching the application's functionality.

Types of Chains

There are many different chains in Langchain that we can use. Here, we are going through three of the fundamental chains.

LLM Chain

The most common chain. Takes an input, formats it, and passes it to an LLM for processing. Basic components are PromptTemplate, an LLM, and an optional output parser.

Sequential Chain

A series of Chains executed in a specific order. There are different variations, including:

- SimpleSequentialChain – singular input/output with output of one step as input to the next
- SequentialChain – allows for multiple inputs/outputs.

Router Chain

The Router Chain is used for complicated tasks. If we have multiple subchains, each of which is specialized for a particular type of input, we could have a router chain that decides which subchain to pass the input to.

Index Chains

These combine your data stored in indexes with LLMs. Useful for question answering over your own documents.

LLM Chain – The simplest chain

The most **basic and widely used chain** in LangChain is the LLMChain.

An **LLMChain** is a simple pipeline consisting of:

- A **PromptTemplate**
- A **Language Model** (like OpenAI's GPT or any ChatModel)
- An optional **OutputParser**

It's used to transform input → format it into a prompt → pass it to an LLM → return the output (optionally parsed).

🔄 How LLMChain Works

- **User Input** is provided (e.g., "solar panels").
- The **PromptTemplate** takes the input and converts it into a prompt string (e.g., "Suggest a brand name for a company that makes solar panels.")
- The **LLM** receives the prompt and generates a response.
- Optionally, an **OutputParser** cleans or structures the result (e.g., JSON output, bullet points, etc.)

🧠 Why Use LLMChain?

- It **abstracts** away prompt creation and model invocation.
- Enables **reusability** of prompt-model workflows.
- Can be **composed** with other chains or tools later.
- Easy to plug into Streamlit, FastAPI, or other LangChain components.

Install Libraries

```
pip install langchain
pip install -qU langchain-google-genai
```

API Key Setup

```
import os
from getpass import getpass
```

```
GOOGLE_API_KEY = getpass("Enter your Gemini API key: ")
os.environ["GOOGLE_API_KEY"] = GOOGLE_API_KEY
```

Import Necessary Libraries

You'll import ChatGoogleGenerativeAI and initialize it with the model name "gemini-1.5-flash":

```
from langchain_google_genai import ChatGoogleGenerativeAI
from langchain.prompts import PromptTemplate
from langchain.chains import LLMChain
```

Initialize Gemini LLM and Prompt Template

We initialize the Gemini LLM with specific parameters, including a temperature of 0.9, which affects the diversity of generated responses.

Furthermore, users must define a **'PromptTemplate'** to input a variable (in this case, "product") and create a **standardized prompt structure**. At runtime, the placeholder '{product}' can be dynamically populated with different product names.

```
llm = ChatGoogleGenerativeAI(
    model="gemini-1.5-flash",  # 👈 Gemini model name
    temperature=0.9)
prompt = PromptTemplate(
    input_variables=["product"],
    template="What is a good name for a company that makes {product}?")
```

Create the LLMChain

We create an instance of the 'LLMChain' class, using a predefined **Gemini Large Language Model and a specified prompt template.** Now, we have the capability to apply the chain to a product such as a **"gaming laptop"** using the chain.run command. This means the chain can dynamically process and generate responses tailored to this specific product input.

```
chain = LLMChain(llm=llm, prompt=prompt, verbose=True)
print(chain.run("gaming laptop"))
```

- Based on this we get the name of a company called **EliteTech Gaming Co.**

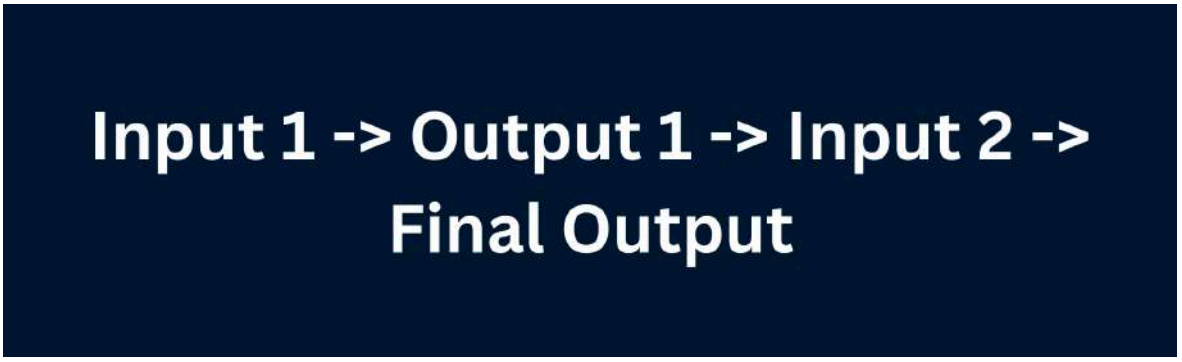
Sequential Chain

A sequential chain is a chain that **combines various individual chains**, where the output of one chain serves as the input for the next in a continuous sequence. It operates by running a series of chains consecutively.

There are two types of sequential chains:

Simple Sequential Chain: which handles a single input and output.

Sequential Chain: manage multiple inputs and outputs simultaneously.



Simple Sequential Chain

🧱 **What it does:**
Runs each step one after another, using the **output of one as input for the next**.

- ✅ **Best for:**
  Simple pipelines like:

a. Summarize → Translate
b. Generate title → Write paragraph

- ❌ **Limitations:**
  You only get the **final output**, not the results of each step.

This straightforward approach is effective when dealing with sub-chains designed for singular inputs and outputs. It ensures a smooth and continuous flow of information, with each step seamlessly passing its output to the subsequent step.

Crafting Chains – Simple Sequential Chains

Simple Sequential Chains allow for a single input to undergo a series of coherent transformations, resulting in a refined output. This sequential approach ensures systematic and efficient handling of data, making it ideal for scenarios where a linear flow of information processing is essential.

Import Required Libraries

These libraries help us build chains using LangChain and connect to the gemini LLMs.

```
# 📥 Step 1: Import Required Libraries
```

```
import os
from getpass import getpass
from langchain_google_genai import ChatGoogleGenerativeAI
from langchain.prompts import ChatPromptTemplate
from langchain.chains import LLMChain, SimpleSequentialChain
```

Setup Gemini API Key

Securely input and set your Gemini API key to authenticate with the gemini service.

```
# 🔐 Step 2: Setup gemini API Key
GOOGLE_API_KEY = getpass("Enter your Gemini API Key: ")
os.environ["GOOGLE_API_KEY"] = GOOGLE_API_KEY
```

Initialize Gemini LLM

We initialize the model ChatGoogleGenerativeAI with a temperature of 0.7 for balanced creativity.

```
# ⚙️ Step 3: Initialize Gemini LLM
llm = ChatGoogleGenerativeAI(
    temperature=0.7,
    model="gemini-1.5-flash"  # Or use gemini-1.5-pro for advanced reasoning
)
```

First Prompt – Company Name from Product

This prompt generates a creative company name based on a given product like "gaming laptop".

```
# ✏️ Step 4: First Prompt - Company Name from Product
first_prompt = ChatPromptTemplate.from_template(
    "What is the best name to describe a company that makes {product}?"
)
chain_one = LLMChain(llm=llm, prompt=first_prompt)
```

Second Prompt – Description from Company Name

This prompt takes the company name generated in Step 4 and creates a short 20-word description.

```
# ✏️ Step 5: Second Prompt - Description from Company Name
second_prompt = ChatPromptTemplate.from_template(
    "Write a 20 words description for the following company: {company_name}"
)
chain_two = LLMChain(llm=llm, prompt=second_prompt)
```

Combine Chains into SimpleSequentialChain

The two chains are connected in sequence so the output of the first becomes input to the second.

```
# 🔗 Step 6: Combine Chains into SimpleSequentialChain
overall_chain = SimpleSequentialChain(
    chains=[chain_one, chain_two],
    verbose=True
)
```

Run the Chain with Input

Provide a product like "gaming laptop" to trigger the sequential generation process.

```
# 🚀 Step 7: Run the Chain with Input
output = overall_chain.run("gaming laptop")
print("\nFinal Output:\n", output)
```

Sequential Chain

- 🧠 **What it does:**
  More advanced. Handles **multiple inputs and outputs** and gives access to **intermediate results**.
- ✅ **Best for:**
  Complex workflows where you need to:

a. Reuse outputs from earlier steps
b. Track every stage of the chain

- 📦 **Bonus:**
  Returns a **dictionary** of all step outputs.

A more general form of sequential chains allows for multiple inputs/outputs. Any step in the chain can take in multiple inputs.

Crafting Chains – Sequential Chains

Import Required Libraries

These are essential to create prompts, connect to Gemini, and build multi-step sequential chains.

```
# 🧱 Step 1: Import Required Libraries
import os
from getpass import getpass
from langchain_google_genai import ChatGoogleGenerativeAI
from langchain.prompts import ChatPromptTemplate
from langchain.chains import LLMChain, SequentialChain
```

Set Up Gemini API Key

Securely input and store your Gemini API key to access their LLMs.

```
# 🔐 Step 2: Set Up Gemini API Key
GOOGLE_API_KEY = getpass("Enter your Gemini API Key: ")
os.environ["GOOGLE_API_KEY"] = GOOGLE_API_KEY
```

Initialize Gemini LLM

We use the Mixtral model with a balanced temperature for creative yet consistent results.

```
# ⚙️ Step 3: Initialize Gemini LLM
llm = ChatGoogleGenerativeAI(
    model="gemini-1.5-flash",  # Or use "gemini-1.5-pro"
    temperature=0.7
)
```

Input Review (in French)

This is the initial text that will go through multiple processing steps.

```
# 📝 Step 4: Input Review (in French)
Review = """Les ordinateurs portables Gamers Tech impressionne par ses
performances exceptionnelles et son design élégant. De sa configuration
matérielle robuste à un clavier RVB personnalisable et un système de
refroidissement efficace, il établit un équilibre parfait entre prouesses
de jeu et portabilité."""
```

Chain 1: Translate Review to English

Takes the input "Review" and returns the output as "English_Review".

```
# 🔹 Chain 1: Translate Review to English
first_prompt = ChatPromptTemplate.from_template(
    "Translate the following review to English:\n\n{Review}"
)
chain_one = LLMChain(llm=llm, prompt=first_prompt, output_key="English_Review")
```

Chain 2: Summarize the English Review

Takes "English_Review" as input and produces a single-sentence "summary".

```
# 🔹 Chain 2: Summarize the English Review
second_prompt = ChatPromptTemplate.from_template(
    "Can you summarize the following review in 1 sentence:\n\n{English_Review}"
)
chain_two = LLMChain(llm=llm, prompt=second_prompt, output_key="summary")
```

Chain 3: Detect Language of Original Review

Accepts "Review" as input and identifies its original language as "language".

```
# 🔹 Chain 3: Detect Language of Original Review
third_prompt = ChatPromptTemplate.from_template(
    "What language is the following review:\n\n{Review}"
)
chain_three = LLMChain(llm=llm, prompt=third_prompt, output_key="language")
```

Chain 4: Generate a Follow-up Message

Takes "summary" and "language" as inputs and generates a localized follow-up response.

```
# 🔹 Chain 4: Generate a Follow-up Message
fourth_prompt = ChatPromptTemplate.from_template(
    "Write a follow-up response to the following summary in the specified language:"
    "\n\nSummary: {summary}\n\nLanguage: {language}"
)
chain_four = LLMChain(llm=llm, prompt=fourth_prompt, output_key="followup_message")
```

Combine All Four Chains

```
# 🔗 Step 5: Combine All Four Chains
# The SequentialChain passes output from one chain to the next with multiple input/output
variables.
overall_chain = SequentialChain(
    chains=[chain_one, chain_two, chain_three, chain_four],
    input_variables=["Review"],
    output_variables=["English_Review", "summary", "followup_message"],
    verbose=True
)
```

Run the Full Multi-Step Chain

```
# 🚀 Step 6: Execute the Chain with the Input Review
# Processes the input through all four chains and returns all intermediate and final results.
output = overall_chain({"Review": Review})
print("\n✅ Final Output:\n")
print(output)
```

What is Langchain Memory?

LangChain Memory is a feature that lets an AI remember things from previous conversations — just like how humans remember what was said earlier in a chat.

Why is it important?

Language models like ChatGPT are usually stateless, meaning they don't remember past messages unless you give them the full history every time.

LangChain Memory solves this by:

- Automatically storing what was said before
- Letting the AI use that memory in future messages to respond more naturally and intelligently

🧠 **What LangChain Memory Does**

🔄 **Before the AI responds:**

- It **reads past conversations** to better understand the current question.

💾 **After the AI responds:**

- It **saves the current exchange** so it can use it in future responses.

❌ **Without Memory:**

You: What's my name?

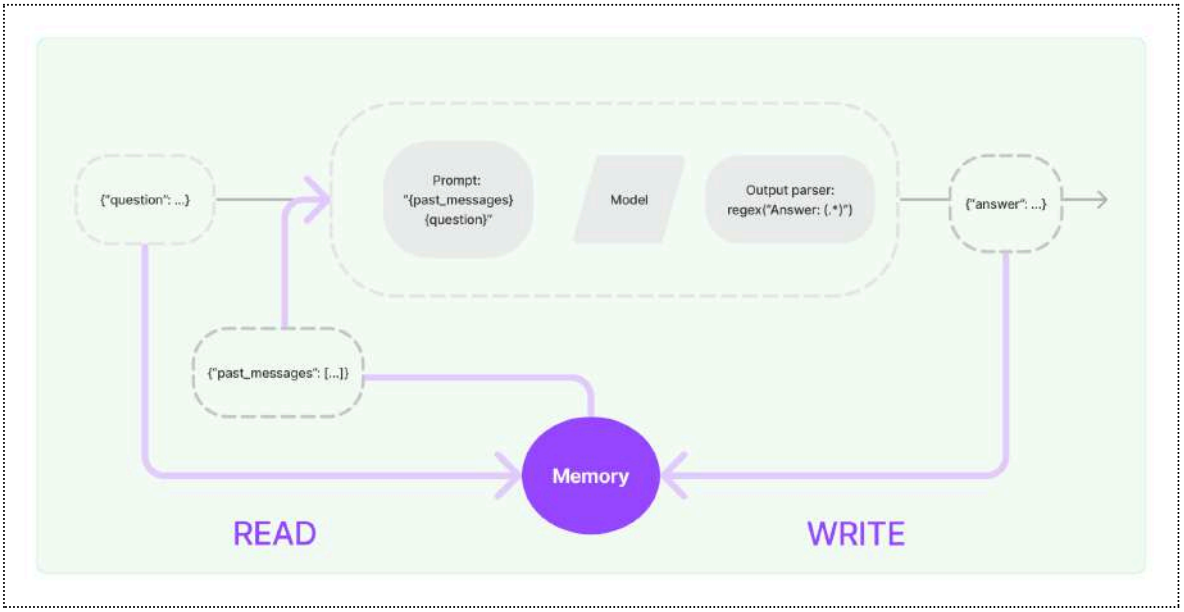
AI: I don't know.

✅ **With Memory:**

You: My name is Sarah. (Later...)

You: What's my name?

AI: Your name is Sarah.



LangChain Memory Types

- **CoversationBufferMemory** ~ Enables the storage of messages and facilitates their extraction into a variable.
- **CoversationBufferWindowMemory** ~ Maintains a list of recent interactions within a conversation, capturing the last 'k' interactions.
- **CoversationTokenBufferMemory** ~Maintains a list of recent interactions based on token length rather than the number of interactions, making it possible to remove older interactions when the token limit is reached.
- **CoversationSummaryMemory** ~ Generates a summary of past interactions within the conversation.

Conversation Buffer Memory

Conversation Buffer Memory is like a notebook where the AI writes down **everything you've said so far** in the chat.

✅ **What it does:**

- Stores **all messages** in order, like a full chat history.
- Helps the AI remember **everything** that has been said earlier in the conversation.
- Makes the AI's replies more accurate and relevant.

⚠️ **What's the downside?**

- If the chat goes on too long, the notebook gets **too full**.
- The AI can only read a limited number of "words" (tokens), so:

a. Responses might get **slower**.
b. The cost of running the model goes **higher**.
c. It might **miss older parts** of the conversation if too long.

🔗 ConversationChain in LangChain

LangChain uses something called a **ConversationChain** to help manage conversations.

What it needs:

1. llm: The language model (like Gemini or GPT)
2. memory: What kind of memory to use (we use ConversationBufferMemory)
3. verbose: Whether to show extra logs/info while chatting

```
# Step 1: Import Required Libraries
import os
from getpass import getpass
from langchain_google_genai import ChatGoogleGenerativeAI
from langchain.chains import ConversationChain
from langchain.memory import ConversationBufferMemory
```

```
# 🔐 Step 2: Setup Gemini API Key
GOOGLE_API_KEY = getpass("Enter your Gemini API Key: ")
os.environ["GOOGLE_API_KEY"] = GOOGLE_API_KEY

# Step 3: Initialize gemini LLM
llm = ChatGoogleGenerativeAI(
    model="gemini-1.5-flash",  # Optional: use "gemini-1.5-pro" for reasoning tasks
    temperature=0.9
)

# Step 4: Initialize Conversation Chain with Full Buffer Memory
conversation = ConversationChain(
    llm=llm,
    verbose=True,
    memory=ConversationBufferMemory()
)
Start Chatting
# Step 5: start chatting
conversation.predict(input="Hi there!")
conversation.predict(input="I'm doing well! Just having a conversation with an AI.")
conversation.predict(input="Tell me about yourself.")
```

For each request, the whole conversation was sent to the API. Above we get an idea of the conversation chain that was ultimately sent over with the final request. This is great to provide the entire previous context with the requests.

ConversationBufferWindowMemory

ConversationBufferWindowMemory is like a smart memory for AI that only remembers the last few messages (not the whole conversation).

✅ Why use it?

- Keeps recent context so the AI stays relevant
- Avoids getting too big or slow
- You can set how many past messages (K) to remember

Think of it like a chat window that only shows the last few messages — the rest are forgotten to keep things fast and focused.

```
#✅ Step 1: Install & Import Required Libraries
from langchain.memory import ConversationBufferWindowMemory
#Step 2: Create Conversation Chain with Windowed Memory
conversation_buffer_window = ConversationChain(
    llm=llm,
    memory=ConversationBufferWindowMemory(k=2),  # Only keep last 2 exchanges
    verbose=True
```

```
)
#✅ Setting k=2 keeps the memory small and efficient
#🔁 Older exchanges are automatically discarded
conversation_buffer_window("Hi There!")
conversation_buffer_window.predict(input="I am an AI enthusiast and love sharing my knowledge through blogs")
conversation_buffer_window.predict(input="I want you to suggest a good and professional name my AI blog page based on my name")
```

Here, we specify the parameter k=2 indicating that we want the last two turns of the conversation to be stored in memory. We can see this effective change in memory compared to above in the sample prompt below

Note: This approach however is not suited for retaining long-term memory but helps us limit the number of tokens being used.

ConversationTokenBufferMemory

ConversationTokenBufferMemory is like a memory that keeps chat history based on the number of words (tokens), not the number of messages.

🔑 **Key points:**

- Keeps recent conversation up to a token limit (like a word count).
- Doesn't summarize — it keeps the exact wording.
- Useful for long chats when you don't want to lose detail but still want to avoid overloading the AI.

Think of it like a box that fits only a certain number of words. Once it's full, old words are removed to make space for new ones.

```
from langchain.memory import ConversationTokenBufferMemory
conversation_with_memory = ConversationChain(
    llm=llm,
    memory=ConversationTokenBufferMemory(llm=llm,max_token_limit=60),
    verbose=True
)
conversation_with_memory.predict(input="Hi, I am Shivan from Almabetter")
conversation_with_memory.predict(input="I am an AI enthusiast and love sharing my knowledge through blogs")
conversation_with_memory.predict(input="I want you to suggest a good and professional name fo my AI blog page based on my name")
```

ConversationSummaryMemory

ConversationSummaryMemory is a smart memory that summarizes the conversation as it goes, instead of storing every message.

✅ **Why use it?**

- Great for long chats without hitting token limits
- Keeps a summary of everything so far, not just recent messages
- Saves space while still remembering the main idea of the conversation

⚠️ **Things to keep in mind:**

- It depends on how well the AI can summarize
- Might use more tokens for short chats, but saves tokens in long ones

```
from langchain.memory import ConversationTokenBufferMemory

conversation_with_memory = ConversationChain(
    llm=llm,
    memory=ConversationTokenBufferMemory(llm=llm,max_token_limit=60),
    verbose=True
)
conversation_with_memory.predict(input="Hi, I am Shivan from Almabetter")

conversation_with_memory.predict(input="I am an AI enthusiast and love sharing my knowledge through blogs")

conversation_with_memory.predict(input="I want you to suggest a good and professional nar for my AI blog page based on my name")
```

Retrieval-Augmented Generation (RAG)

What is RAG?

RAG stands for **Retrieval-Augmented Generation**.
It's a method where AI models **fetch useful information from external sources** (like documents, databases, or websites) and use that information to generate **more accurate and context-aware answers**.

Instead of only relying on pre-trained knowledge, RAG:

1. **Retrieves** relevant documents for a given query.
2. **Generates** responses using both the query and retrieved documents.

This makes responses factual, updated, and trustworthy.

Why Use RAG?

Traditional LLMs (like GPT) have a fixed knowledge base. They can't access real-time or external data. RAG solves this by:

- 🧠 **Fresh Knowledge**: Pulls information from external sources (documents, APIs, etc.)
- 🔍 **Specific Context**: Retrieves content relevant to the user's query
- 🚫 **Fewer Hallucinations**: Grounds responses in actual documents
- ✅ **Trustworthy**: Produces explainable, fact-based answers

In short: **RAG = Retrieval (find info) + Generation (answer using it).**

How RAG Works – Architecture

RAG uses a **2-step pipeline**:

- **Retriever**

a. Uses semantic search (FAISS, BM25, Elasticsearch, etc.)
b. Converts query → embedding vector → finds similar vectors from a document store

- **Generator**

a. A language model (e.g., GPT, BART, T5)
b. Takes **query + retrieved documents** → generates final answer

Formula:

```
response = Generator(query + retrieved_documents)
```

Pipeline Flow:
User Query → Retriever → Top-k Docs → Generator → Final Answer

Example

Query: "What is quantum computing?"

- **Retriever**: Finds top 3 Wikipedia snippets on quantum computing.
- **Generator**: Uses those snippets + query → produces fact-based explanation.

Real-World Use Cases

- 🤖 **Chatbots with Company Documents** – Answer employee/customer questions using manuals, policies, onboarding docs.
- ⚖️ **Legal & Finance Q&A Bots** – Summarize contracts, regulations, financial reports.
- 🧪 **Research Assistants** – Retrieve & summarize academic or medical papers.

- 💬 **Customer Support Agents** – Pull data from FAQs/knowledge bases.
- 🎯 **Personalized Recommendations** – Suggest products, books, or articles by combining user history + external data.

Vector Databases

A **Vector Database** stores high-dimensional vectors (numerical embeddings of text, images, audio, etc.).

Why they matter:

- 🔍 Enable **semantic search** (find meaning, not just keywords)
- ⚡ Support **fast similarity search** with approximate nearest neighbors (ANN)
- 🤖 Power **chatbots, search engines, recommendations**

Popular Vector Databases:

- **Chroma DB** – Easy, lightweight, for prototypes/small projects
- **FAISS** – Open-source, offline/local vector similarity search (by Meta)
- **Pinecone** – Managed, scalable, production-ready
- **Qdrant** – Open-source, fast, metadata filtering
- **Redis (with Vector Search)** – Real-time apps, caching + vector search
- **Weaviate** – Built-in ML modules, all-in-one semantic search solution

Essential RAG Components

- **Documents / Knowledge Base**

a. The raw information source (PDFs, manuals, research papers).
b. Example: A company's 50+ product guides → customer queries are answered using them.

- **Chunking**

a. Splitting large documents into smaller parts (e.g., 200–300 words).
b. Makes retrieval more efficient.
c. Analogy: Like slicing a big cake into smaller pieces 🍰.

- **Embedding Model**

a. Converts text chunks into vectors (semantic meaning).
b. Example:
   - "AI will transform the world" → [0.12, -0.98, 0.45, 0.67...]
c. Tools: OpenAI's text-embedding-ada-002, HuggingFace SentenceTransformers, Cohere.

- **Similarity Search**

a. Converts query → vector → compares with stored vectors.
b. Finds semantically similar chunks, even if words differ.
c. Methods: **Cosine similarity** (most common), dot product, Euclidean distance.

✅ **In summary**:
RAG combines the **power of retrieval** (finding the right documents) with the **creativity of generation** (LLMs) → producing **accurate, real-time, and context-aware answers**.

Lesson 4 : MultiPDFChat: Document Interaction Simplified

What is Rag?

RAG stands for **Retrieval-Augmented Generation**.

It is a technique where a language model (like ChatGPT or Gemini) first **retrieves relevant information** from documents or databases, and then **generates an answer** using that information.

In simple terms: RAG = Search + Generate

Why RAG?

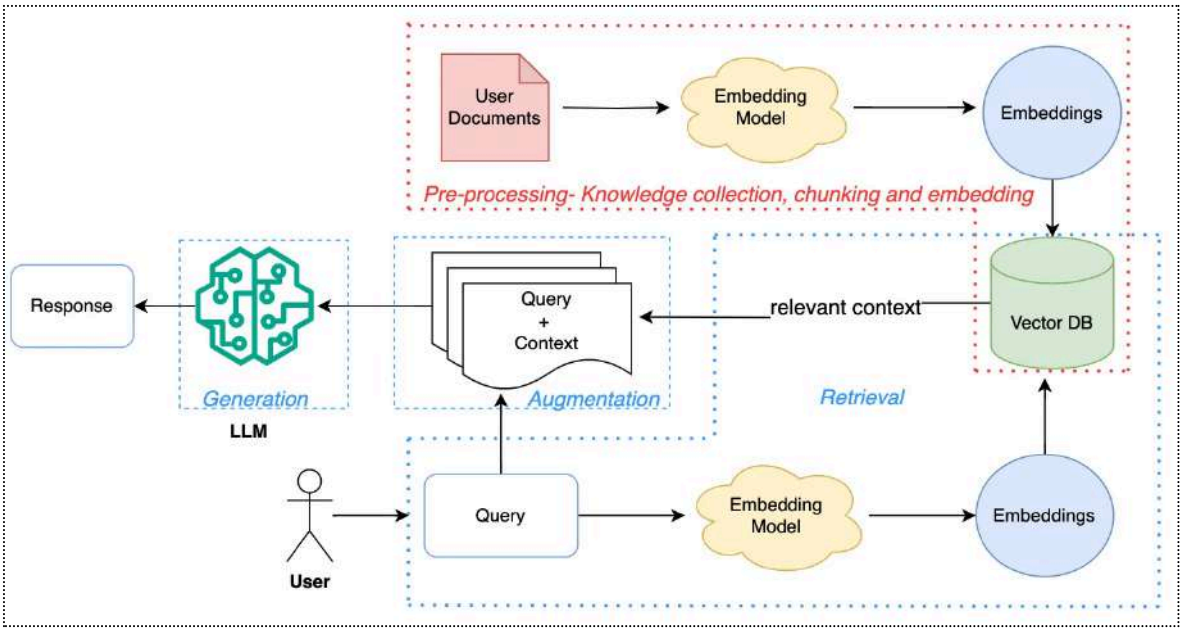
Traditional language models are limited to what they learned during training. If you want to answer based on **external or private knowledge (like PDFs, docs, wikis) without fine-tuning, RAG is the answer.**

RAG = Retrieval + Generation = Smart, Factual, and Custom LLM Responses

RAG (Retrieval-Augmented Generation) Architecture – A Deep Dive with Visual Flow

Components of Rag

- Ingestion / Indexing
- Retrieval
- Generation



📷 **Simple Use Case**

"Ask your company handbook anything!"

Imagine an HR chatbot that uses your company's documents to answer questions like:
"How many casual leaves do I get?"

Key Components in the RAG Architecture

Pre-processing (Indexing Phase)

🎯 **Goal**: Convert raw HR documents into searchable vector representations.

- **User Documents**

  HR Handbook, Leave Policy PDF, HR FAQs

- **Embedding Model**
  → Converts chunks of these documents into vector format
- **Embeddings**
  → Each chunk becomes a numeric vector that captures its meaning
- **Vector DB**
  → All embeddings are stored here for fast semantic search

✅ This step prepares your HR documents so the system can later search for relevant answers.

Retrieval Phase

🎯 **Goal**: Find the most relevant document chunks that can answer the user's query.

- **User Input**
  → *"How many casual leaves do I get?"*
- **Query Embedding**
  → The system turns this question into a vector using the same embedding model
- **Vector Search**
  → Compares the question vector with all document vectors in the Vector DB
- **Relevant Context**
  → Retrieves chunks like:
  "Employees are entitled to 12 casual leaves per calendar year..."

Augmentation Phase

🎯 **Goal**: Add useful context to the user's question before passing it to the AI model.

- Combine the **query + retrieved context**
- Example Input to LLM:
  Query: "How many casual leaves do I get?"
  Context: "As per the leave policy, employees are eligible for 12 casual leaves annually..."

Generation Phase

🎯 **Goal**: Use the combined input to generate a clear and helpful answer.

- The **LLM (like GPT)** receives the query + context
- It generates the final response:
  "You are eligible for 12 casual leaves per year as per company policy."
- The response is sent back to the user

What is a RAG Chain?

The **RAG Chain** is the **core runtime workflow** in a Retrieval-Augmented Generation (RAG) system.

It activates **after data is indexed**, and is responsible for:

- Retrieving relevant information from the vector database
- Augmenting the user query with retrieved context
- Generating a response using a Large Language Model (LLM)

⚙️ **How the RAG Chain Works (Step-by-Step)**

Example Query: "How many casual leaves do I get?"

1. **Query Embedding**
- The user's question is converted into a vector using the **same embedding model** used during indexing.
2. **Vector Search**
- The system searches the **Vector Database** to find similar document chunks.

3. **Retrieve Context**
- The top relevant chunks (e.g., from HR policy) are retrieved.
4. **Augmentation**
- The system combines:
  - The original query
  - The retrieved content
    → into a single prompt
5. **LLM Generation**
- The combined input is sent to the **LLM** (e.g., GPT-4, Gemini)
- It generates a fact-based, context-aware response
6. **Return Answer to User**
- *Example output: "You are eligible for 12 casual leaves per year."*

Project Title: MultiPDFChat – Document Interaction Simplified

🎯 Goal

To build a smart chatbot that allows users to **ask questions across multiple PDF documents** and receive accurate, real-time answers using **Retrieval-Augmented Generation (RAG)**.

⚙️ How the RAG Chain Works

- **User Uploads PDFs**
  → Multiple documents are uploaded and stored for processing.
- **Document Chunking**
  → Each PDF is split into manageable text chunks.
- **Embedding Generation**
  → Use an embedding model (e.g., text-embedding-ada-002) to convert chunks into vectors.
- **Indexing in Vector DB**
  → Store these embeddings in a vector database (e.g., FAISS, ChromaDB).
- **User Enters a Query**
  → User types a question like: *"What is the refund policy mentioned in any PDF?"*
- **Query Embedding**
  → The question is embedded using the same embedding model.
- **Similarity Search**
  → Perform a vector search to retrieve the most relevant document chunks.
- **Query + Context Augmentation**
  → Combine user query with top-matching chunks.
- **LLM Generation**
  → Pass the combined input to an LLM (e.g., GPT-4 or Gemini) to generate the final answer.
- **Response Delivery**
  → Display the result to the user in a chat-like interface.

Link to Datasets

PDF-1: https://drive.google.com/file/d/1e9Hl5nFltX93Ob4DU875QeB4Uj1eW_p6/view?usp=sharing

PDF-2: https://drive.google.com/file/d/1gEf9uKyMe1XtftGHTxDg_kREK40mWRQ9/view?usp=sharing

STEP 1: Install Required Libraries

Installs core libraries for:

- PDF reading (PyPDF2)
- LangChain framework
- Google Gemini for embeddings and chat
- FAISS for vector similarity search

```
!pip install PyPDF2 langchain langchain_google_genai google-generativeai faiss-cpu

!pip install -U langchain-community
```

STEP 2: Import Libraries

Load tools for:

- Reading PDFs
- Splitting text
- Generating embeddings
- Chatting with Gemini
- Building RAG chains

```
# Import necessary libraries
from PyPDF2 import PdfReader  # For reading PDF files
from langchain.text_splitter import RecursiveCharacterTextSplitter  # For splittin
text into manageable chunks
import os  # For interacting with the operating system
from langchain_google_genai import GoogleGenerativeAIEmbeddings  # For generating
embeddings using Google Generative AI
import google.generativeai as genai  # For using Google's generative AI capabiliti
from langchain_community.vectorstores import FAISS
  # For efficient similarity search with embeddings
from langchain_google_genai import ChatGoogleGenerativeAI  # For chat-based
interactions with Google AI
from langchain.chains.question_answering import load_qa_chain  # For loading QA
chains
from langchain.prompts import PromptTemplate  # For creating templates for prompts
from google.colab import drive  # For mounting Google Drive in Colab
```

STEP 3: Mount Google Drive (for saving the vector index)

Mounts your Google Drive to read or save files (like FAISS index).

```
# Mount Google Drive
drive.mount('/content/drive')
```

STEP 4: Set Gemini API Key

Authorizes access to Google Gemini services using your API key.

```
# Set your Google API key
os.environ["GOOGLE_API_KEY"] = "GOOGLE_API_KEY" # Paste Your Gemini API Key
genai.configure(api_key=os.environ["GOOGLE_API_KEY"])
```

STEP 5: Extract Text from PDFs

Reads multiple PDFs and returns their combined text content.

```
def get_pdf_text(pdf_paths):
    text = ""
    for pdf_path in pdf_paths:
        pdf_reader = PdfReader(pdf_path) # Create a PdfReader object for the curre
PDF file
        for page in pdf_reader.pages: # Iterate through each page in the PDF
            text += page.extract_text() # Extract text from the page and append it
to the 'text' variable
    return text # Return the concatenated text from all PDFs
```

Extracts text from a list of PDF files.

Args:
pdf_paths (list): A list of paths to PDF files.

Returns:
str: A concatenated string containing the text extracted from all the specified PDF files.

This function iterates over each PDF file provided in the 'pdf_paths' list,
reads the content of each page, and appends the extracted text to a single string.

STEP 6: Split Text into Chunks

Splits long text into overlapping chunks to preserve context (important for QA).

```
def get_text_chunks(text):
```

```
    text_splitter = RecursiveCharacterTextSplitter(chunk_size=10000,
chunk_overlap=1000)
    # Initialize the RecursiveCharacterTextSplitter with a chunk size of 10,000
characters
    # overlap of 1,000 characters between consecutive chunks to maintain context.
    chunks = text_splitter.split_text(text) # Split the text into chunks
    return chunks # Return the list of text chunks

    # I love apple -> s1
    # Wow, Even i love apple -> s2
```

Splits the input text into manageable chunks

Args:
text (str): The input text to be split into chunks.

Returns:
list: A list of text chunks, each with a maximum size defined by 'chunk_size',
and overlapping content defined by 'chunk_overlap'.

This function utilizes the RecursiveCharacterTextSplitter to divide the provided text
into smaller segments. Each chunk can be up to 'chunk_size' characters long,
with a specified overlap of 'chunk_overlap' characters between consecutive chunks.
This is useful for processing large texts without losing context.

STEP 7: Create Embeddings & Store in FAISS

Converts chunks to embeddings using Gemini.

Stores them in FAISS for similarity search.

```
# Document Ingestion and Embedding

def get_vector_store(text_chunks):
    embeddings = GoogleGenerativeAIEmbeddings(model="models/embedding-001") # Crea
embeddings using the specified AI model
    vector_store = FAISS.from_texts(text_chunks, embedding=embeddings) # Create a
FAISS vector store from the text chunks
    vector_store.save_local("/content/drive/MyDrive/faiss_index")  # Save the vect
store locally for future use
```

This function takes text chunks from the PDFs, creates embeddings using Google's AI model, and stores these embeddings in a FAISS vector store. This is the "indexing" part of RAG, where documents are prepared for efficient retrieval.

Args:
text_chunks (list): A list of text chunks to be converted into embeddings.

Returns:
FAISS: A FAISS vector store containing the embeddings of the provided text chunks.

This function uses Google's AI model to generate embeddings for the provided text chunks, which are then stored in a FAISS vector store. This setup is essential for retrieval-augmented generation (RAG) workflows, allowing for quick and efficient document retrieval based on embeddings.

STEP 8: Create Conversational Chain (QA logic)

Defines how to answer questions using:

- Retrieved documents
- Gemini LLM with custom prompt

```
def get_conversational_chain():
    prompt_template = """
    Answer the question as detailed as possible from the provided context, make su
to provide all the details, if the answer is not in
    provided context just say, "answer is not available in the context", don't
provide the wrong answer\n\n
    Context:\n {context}?\n
    Question: \n{question}\n
    Answer:
    """
    model = ChatGoogleGenerativeAI(model="gemini-1.5-flash", temperature=0.3)  #
Initialize the AI model with specified parameters
    prompt = PromptTemplate(template=prompt_template, input_variables=["context",
"question"]) # Create a prompt template

    chain = load_qa_chain(model, chain_type="stuff", prompt=prompt) # Load the QA
chain with the model and prompt
    return chain  # Return the configured question-answering chain
```

Constructs a conversational chain for question-answering based on a given context.

Returns:
chain: A configured question-answering chain using Google’s generative AI model.

This function defines a prompt template that guides the AI to provide detailed answers based on the context provided. If the answer is not found within the context, the AI will respond with "answer is not available in the context." It initializes the ChatGoogleGenerativeAI model and sets a temperature for response variability, then creates a PromptTemplate using the specified template and input variables. Finally, it loads and returns a question-answering chain configured with the model and prompt.

STEP 9: Handle User Questions

- Retrieves relevant chunks from FAISS using the user's question
- Sends it to Gemini with the prompt template
- Prints the AI-generated answer

```
# Retrieval: In the user_input function
def user_input(user_question):
    # Retrieval: Perform a similarity search on the vector store
    embeddings = GoogleGenerativeAIEmbeddings(model="models/embedding-001")  #
Create embeddings for the search

    new_db = FAISS.load_local("/content/drive/MyDrive/faiss_index", embeddings,
allow_dangerous_deserialization=True) # Load the FAISS vector store
    docs = new_db.similarity_search(user_question) # Retrieve the most relevant
documents based on the user's question

    # Generation: Generate a response using the conversational chain

    chain = get_conversational_chain() # Get the configured conversational chain

    response = chain(
        {"input_documents": docs, "question": user_question}, # Pass retrieved doc
and question to the chain
        return_only_outputs=True) # Return only the output from the chain
    print(response) # Print the full response object for debugging
    print("Reply: ", response["output_text"]) # Print the generated reply text
```

When a user asks a question, the system performs a similarity search on the vector store to retrieve the most relevant documents. This is the "retrieval" part of RAG.

Args:
  user_question (str): The question posed by the user.

This function implements the retrieval-augmented generation (RAG) approach:
1. **Retrieval**: It performs a similarity search on the vector store to find the most relevant documents related to the user's question.
2. **Generation**: It then generates a response using a language model based on the retrieved documents and the user's question.
The retrieved documents are then passed along with the user's question to a language model (in this case, Google's Gemini model) to generate a response. This is the "generation" part of RAG.

STEP 10: Main Orchestration Function

Controls the full pipeline:

- Loads and processes PDF

- Creates embeddings and stores them
- Repeatedly accepts user questions and answers them using RAG

```
import os

def main():

    print("Chat with MultiPDF File using Gemini Model💁")

    # Define paths to the PDF files to be processed
    pdf_paths = ['/content/pdf1.pdf',
                 '/content/pdf2.pdf']
    # Specify the path for the FAISS index
    index_path = "/content/drive/faiss_index"

    if not os.path.exists(index_path):
        print("Processing PDFs and creating index...")
        raw_text = get_pdf_text(pdf_paths) # Extract text from the PDF files
        text_chunks = get_text_chunks(raw_text)  # Split the extracted text into
chunks
        get_vector_store(text_chunks) # Create a vector store from the text chunks
        print("Index created.")
    else:
        print("Using existing index.")

    # Interactive loop for user questions
    while True:
        user_question = input("Ask a question from the PDF files (or type 'exit' t
quit): ")
        if user_question.lower() == 'exit':
            break # Exit the loop if the user types 'exit'
        user_input(user_question) # Process the user's question

if __name__ == "__main__":
    main() # Run the main function when the script is executed
```

Main function to interact with PDFs using the Gemini AI model.

This function orchestrates the process of creating a vector store from PDF files,
allowing users to ask questions and receive answers based on the content of those PDFs.

Multi-PDF Chatbot with Streamlit + Gemini

Step-by-Step Summary

1. **Import all required libraries**

Includes **Streamlit**, **LangChain**, **Google Gemini**, **PyPDF2**, and **FAISS**.

2. **Set the Google Gemini API key**

   Use **environment variables** to configure access securely.

3. **Define** `get_pdf_text()`
- Extracts text from uploaded **PDF files** using **PyPDF2**.
4. **Define** `get_text_chunks()`
- Splits the extracted text into **manageable chunks** for embedding using **RecursiveCharacterTextSplitter**.
5. **Define** `get_vector_store()`
- Creates **embeddings** using **Gemini Embeddings API** and saves them using **FAISS**.
6. **Define** `get_conversational_chain()`
- Loads a **LangChain QA chain** with **Gemini Pro LLM** and a **custom prompt template**.
7. **Define** `chat_with_pdf()`
- Performs **similarity search** over stored embeddings and generates a response using the LLM **chain**.
8. **Set up the Streamlit UI**
- Add app **title** and create a **sidebar** for uploading PDFs.
9. **Handle file uploads and processing**
- On button click, save uploaded PDFs, extract text, split it, and generate vector store embeddings.
10. **Use session state for chat history**
- Store past **questions and answers** using `st.session_state`.
11. **Capture user input**
- Display a text input field for users to **enter questions**.
12. **On submit, search and generate answer**
- Embed the question, retrieve relevant chunks, and generate a **Gemini-powered answer**.
13. **Display chat history**
- Show all previous **Q&A pairs** in reverse order (latest first) using **Streamlit markdown**.

✅ Step-by-Step Instructions to create and Run the Streamlit + Gemini PDF Chatbot

1. **Open VS Code**
2. **Open Project Folder** : Go to **File** → **Open Folder** and select (or create) the folder for your project.
3. **Create the** `app.py` **File** :
   Create a new file named `app.py` and paste your **Streamlit chatbot code** into it.
4. **Create a requirements.txt File**
5. **Create a Virtual Environment** :
   Open the terminal in VS Code and run:
- **env:** python -m venv env
- **Windows** .\env\Scripts\activate
- **Mac** source env/bin/activate
6. **Install All Dependencies**

```
pip install -r requirements.txt
```

7. **Run the Streamlit App**

```
streamlit run app.py
```

Create requirements.txt file

```
streamlit
PyPDF2
langchain
langchain_community
langchain_google_genai
google-generativeai
faiss-cpu
```

Create app.py (Streamlit App Code)

```
# Import necessary libraries
import os
import streamlit as st
from PyPDF2 import PdfReader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_google_genai import GoogleGenerativeAIEmbeddings,
ChatGoogleGenerativeAI
import google.generativeai as genai
from langchain_community.vectorstores import FAISS
from langchain.chains.question_answering import load_qa_chain
from langchain.prompts import PromptTemplate

# ----------------------------
# Set up Google Gemini API key
# ----------------------------
# It's recommended to use secrets or environment variables in production
os.environ["GOOGLE_API_KEY"] = "GOOGLE_API_KEY"
genai.configure(api_key=os.environ["GOOGLE_API_KEY"])

# -----------------------------------
# Function: Extract text from PDF files
# -----------------------------------
def get_pdf_text(pdf_paths):
    text = ""
    for path in pdf_paths:
        pdf_reader = PdfReader(path)
        for page in pdf_reader.pages:
            text += page.extract_text() or ""  # Handle cases where text may be No
    return text

# -------------------------------------------------
# Function: Split text into manageable chunks
# -------------------------------------------------
```

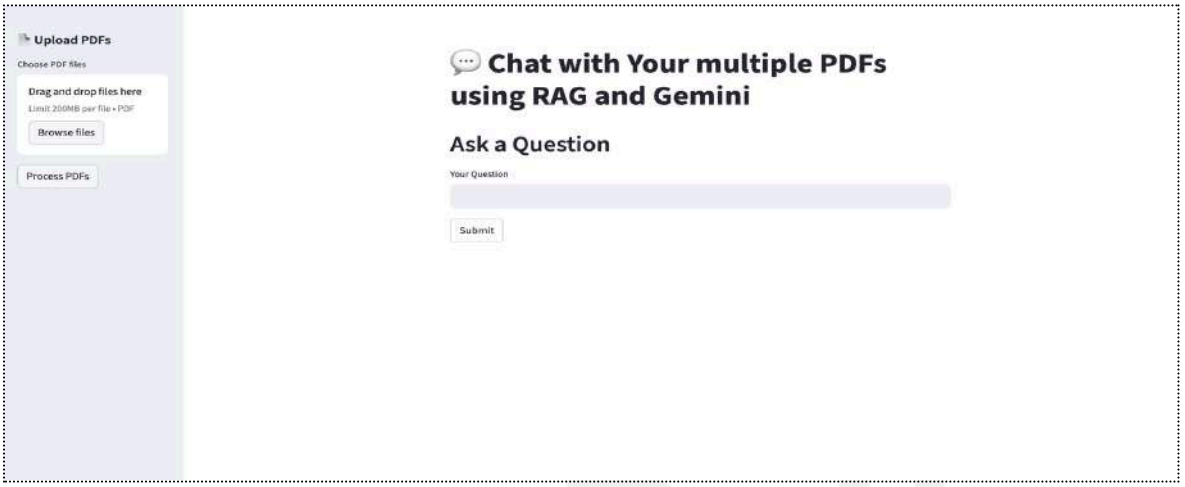
```
def get_text_chunks(text):
    splitter = RecursiveCharacterTextSplitter(chunk_size=10000, chunk_overlap=1000
    return splitter.split_text(text)

# ------------------------------------------------------------------
# Function: Generate and save vector embeddings using FAISS and Gemini
# ------------------------------------------------------------------
def get_vector_store(text_chunks):
    embeddings = GoogleGenerativeAIEmbeddings(model="models/embedding-001")
    vector_store = FAISS.from_texts(text_chunks, embedding=embeddings)
    vector_store.save_local("faiss_index")  # Saves the vector store locally

# ------------------------------------------------------------------
# Function: Load the conversational chain with custom prompt template
# ------------------------------------------------------------------
def get_conversational_chain():
    prompt_template = """
    Answer the question as detailed as possible from the provided context.
    If the answer is not in the context, say "answer is not available in the
context".

    Context:\n{context}\n
    Question:\n{question}\n
    Answer:
    """
    # Initialize the Gemini Pro chat model
    model = ChatGoogleGenerativeAI(model="gemini-1.5-flash", temperature=0.3)
    # Create a custom prompt
    prompt = PromptTemplate(template=prompt_template, input_variables=["context",
"question"])
    # Load a simple Q&A chain with the model and prompt
    return load_qa_chain(model, chain_type="stuff", prompt=prompt)

# ------------------------------------------------------------------
# Function: Search PDFs and answer user question using embeddings
# ------------------------------------------------------------------
def chat_with_pdf(question):
    # Load saved embeddings
    embeddings = GoogleGenerativeAIEmbeddings(model="models/embedding-001")
    db = FAISS.load_local("faiss_index", embeddings,
allow_dangerous_deserialization=True)
    # Perform similarity search on question
    docs = db.similarity_search(question)
    # Get the conversational chain
    chain = get_conversational_chain()
    # Run the chain with documents and question
    result = chain({"input_documents": docs, "question": question},
return_only_outputs=True)
```

```
    return result["output_text"]

# ----------------------
# Streamlit Web Interface
# ----------------------

# App Title
st.title("💬 Chat with Your multiple PDFs using RAG and Gemini")

# Sidebar: File Upload Section
with st.sidebar:
    st.header("📄 Upload PDFs")
    uploaded_files = st.file_uploader("Choose PDF files", type="pdf",
accept_multiple_files=True)

    # Process PDFs and create vector store on button click
    if st.button("Process PDFs") and uploaded_files:
        file_paths = []
        for file in uploaded_files:
            # Save uploaded files temporarily
            with open(file.name, "wb") as f:
                f.write(file.read())
                file_paths.append(file.name)

        # Extract text, split, and generate vector store
        raw_text = get_pdf_text(file_paths)
        chunks = get_text_chunks(raw_text)
        get_vector_store(chunks)

        st.success("✅ PDFs processed and indexed successfully!")

# Chat history management
if "history" not in st.session_state:
    st.session_state.history = []

# Main chat section
st.header("Ask a Question")
question = st.text_input("Your Question")  # User input

# Handle submit button
if st.button("Submit") and question:
    # Get answer using chat_with_pdf
    answer = chat_with_pdf(question)
    # Store Q&A in session history
    st.session_state.history.append((question, answer))

# Display chat history (latest first)
for q, a in st.session_state.history[::-1]:
```

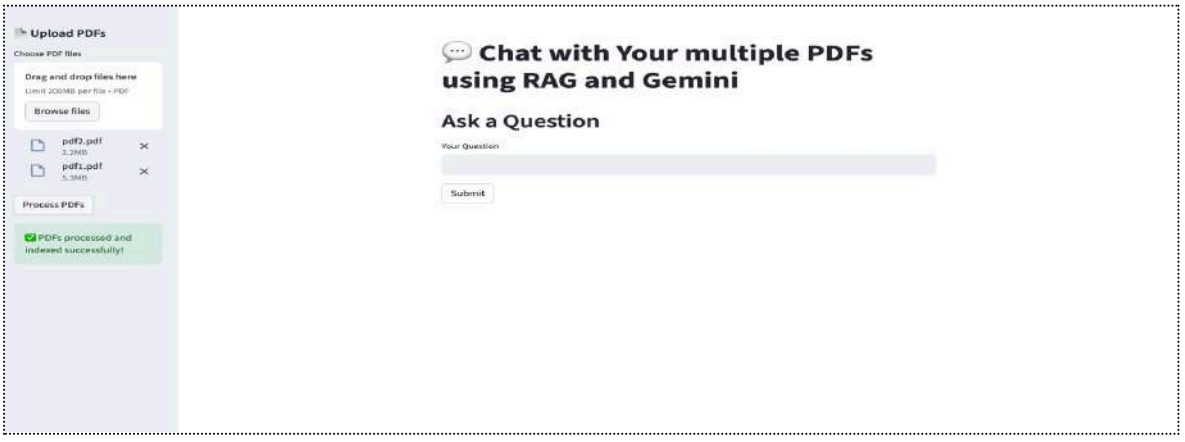
```
    st.markdown(f"**Q:** {q}")
    st.markdown(f"**A:** {a}")
```

Writing app.py

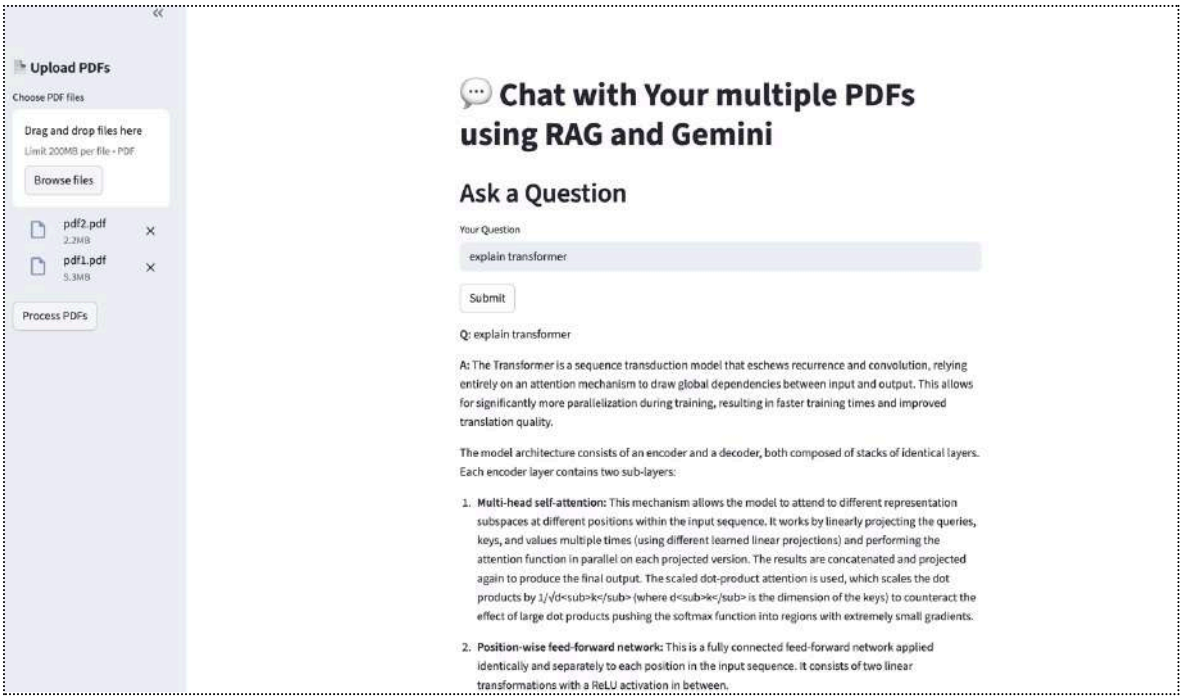
App looks like this



Upload pdf1 and pdf2 and click on process



Ask any question related to pdf


Upload PDFs
Choose PDF files
Drag and drop files here
Limit 200MB per file • PDF
Browse files
pdf2.pdf
2.2MB
pdf1.pdf
5.3MB
Process PDFs
Chat with Your multiple PDFs using RAG and Gemini
Ask a Question
Your Question
explain transformer
Submit
Q: explain transformer
A: The Transformer is a sequence transduction model that eschews recurrence and convolution, relying entirely on an attention mechanism to draw global dependencies between input and output. This allows for significantly more parallelization during training, resulting in faster training times and improved translation quality.
The model architecture consists of an encoder and a decoder, both composed of stacks of identical layers. Each encoder layer contains two sub-layers:
1. Multi-head self-attention: This mechanism allows the model to attend to different representation subspaces at different positions within the input sequence. It works by linearly projecting the queries, keys, and values multiple times (using different learned linear projections) and performing the attention function in parallel on each projected version. The results are concatenated and projected again to produce the final output. The scaled dot-product attention is used, which scales the dot products by 1/√d_k (where d_k is the dimension of the keys) to counteract the effect of large dot products pushing the softmax function into regions with extremely small gradients.
2. Position-wise feed-forward network: This is a fully connected feed-forward network applied identically and separately to each position in the input sequence. It consists of two linear transformations with a ReLU activation in between.

Lesson 5: Agentic AI & AI Agents Using n8n

What Is an Agent?

An **Agent** is like a helper that can observe something and then act based on what it sees.

Think of it as a **smart assistant** that:

- Listens or watches for events (like receiving an email)
- Understands what just happened
- Takes action (like replying or saving the info)

🛠️ **Real-Life Analogy:**

A **smart thermostat** that senses the temperature and turns the AC on/off automatically.

Interesting Applications:

- **Home Automation**: Turn on lights when it's dark
- **Reminder Bots**: Ping you if calendar is empty for the day
- **Support Triage**: Automatically tag incoming support ticket

What Is an AI Agent?

An **AI Agent** is a smarter agent that uses AI to understand, decide, and act.

It's like giving **ChatGPT a to-do list and a body**.

It can:

- Read information (like a resume or a job listing)
- Think and compare
- Take actions (like selecting best jobs, replying to emails)

Real-Life Example:

AI Agent reads your resume, searches job listings, picks the best ones, and sends you a shortlist with reasons.

Interesting Applications:

- **Job Search Buddy**: Filters job posts that match your skills
- **Email Classifier**: Organizes inbox using AI
- **Lead Qualifier**: Reads CRM notes and labels hot leads

What Is Agentic AI?

Agentic AI is an AI that doesn't just answer you — it **plans, decides, and acts on its own**.

You give it a goal like "apply to 5 jobs this week," and it figures out:

- Where to find them
- How to compare with your resume
- What to send

- When to send it

It may even retry if one step fails.

Analogy:

A **project intern** who does the entire task and only updates you when done — no need to micromanage.

Types of Agentic AI:

1. **Reactive Agents**: These agents respond to their environment but don't store past interactions. They follow predefined rules or patterns.
a. **Example**: A basic thermostat that reacts to temperature changes without learning from previous interactions.
2. **Deliberative Agents**: These agents reason and plan actions based on current and past states of the environment. They often involve more complex decision-making processes.
a. **Example**: Autonomous vehicles that navigate roads by making decisions based on surrounding objects and past experiences.
3. **Learning Agents**: These agents use machine learning techniques to improve their performance over time. They learn from data, feedback, or environmental interaction.
a. **Example**: Personalized recommendation systems that adapt to users' preferences.

Interesting Applications:

- **Weekly AI Recruiter**: Finds and applies for jobs on your behalf
- **Smart Travel Planner**: Plans itineraries based on calendar, books hotels
- **Personal Finance Assistant**: Tracks expenses, notifies unusual spending, suggests savings

Top Differences: AI Agent vs Agentic AI vs RAG

Feature / Question	🤖 AI Agent	⚡ Agentic AI	📚 RAG (Retrieval-Augmented Generation)
Who takes the lea	You give instructions	It takes initiative on its own	You ask a question
Type of task hand	One-step tasks (e.g., reply summarize)	Multi-step tasks (plan → act follow-up)	Question-answering using extern documents
Does it plan or jus react?	Just reacts to input	Plans, decides, and execute	Reacts to input with smarter info
Autonomy	Low – needs prompt every time	High – works without const supervision	No autonomy – depends on question
Can it use tools/APIs?	Sometimes	Yes – uses many tools	Only retrieves data for better LL answers
Main goal	Perform a given AI task	Complete a goal end-to-end	Give accurate answers using retrieved info

Example	“Summarize this email”	“Find and apply to 5 jobs for this week”	“What is the refund policy in this 100-page PDF?”

Analogy to Understand the Difference

- 🧠 AI Agent A smart employee waiting for you to assign tasks. Example: "Summarize this email."
- ⚡ Agentic AI A proactive assistant who sees your to-do list and does it before you ask. Example: "Apply to 5 jobs weekly for me."
- 📚 RAG A librarian that finds the right document pages to help ChatGPT give a smarter answer. Example: "What’s in my PDF?"

n8n – Build Smart AI Agents Visually

n8n (pronounced "n-eight-n") is an **open-source workflow automation tool**.
It lets you **connect apps, APIs, and AI tools** — all using a **drag-and-drop interface**.

Think of it like a **visual programming tool** for building bots, agents, and automations — no deep coding needed!

❓ Why Use n8n?

Benefit	Description
🧠 **Low-code**	Build complex logic visually without coding
🔗 **App connectors**	Has 350+ built-in nodes (Gmail, Telegram, Notion, OpenAI, etc.)
⚙️ **Custom workflows**	You control the logic, conditions, loops, and triggers
🌐 **API ready**	Easily connect to any REST API or webhook
💡 **AI friendly**	Works with OpenAI, Gemini, HuggingFace, etc. to integrate LLMs into flows
🔄 **Runs locally or in cloud**	Great for privacy and flexibility

💼 Real-World Applications of n8n

Use Case	Description
📩 Email Summarizer Agent	Fetches email → Uses AI to summarize → Sends you the summary on Telegram
📄 PDF QnA Assistant	Upload PDF → Ask questions → AI responds using content
💼 Job Match Agent	Scrapes job posts → Compares with your resume → Alerts you with matches

📊 Auto Report Generator	Pulls data → Summarizes with AI → Sends report daily via email
🧑‍💻 Customer Support Triage Agent	Reads incoming tickets → Uses AI to categorize and assign right agent
💬 WhatsApp Chatbot with Memory	Receives user input → Uses AI and memory to answer → Logs chat to Notion

🚀 How to Sign Up for n8n Cloud

Follow these steps to sign up for **n8n Cloud**:

1. **Go to the n8n Cloud Website**:
a. Visit the official **n8n Cloud** website at https://n8n.io/cloud.
2. **Click on "Get Started for Free"**:
a. On the homepage, click the **"Get Started for Free"** button to begin the sign-up process.
3. **Create an Account**:
a. You'll be prompted to create an account.
b. You can sign up using your **email** or choose to sign up with a third-party service like **Google** or **GitHub**.
4. **Enter Your Information**:
a. Provide your **email address** and **password**, or select the appropriate third-party login option.
b. Click **Sign Up** to proceed.
5. **Verify Your Email**:
a. After signing up, n8n will send a **verification email** to your provided email address.
b. Open the email and click on the **verification link** to confirm your account.
6. **Complete Setup**:
a. Once your email is verified, return to the n8n Cloud dashboard.
b. Complete any additional setup, such as configuring your workspace, billing (if applicable), and starting your first workflow.
7. **Start Using n8n Cloud**:
a. You are now ready to create workflows and automate processes in **n8n Cloud**.

Benefits of n8n Cloud:

- **No Setup Required**: No need to install or maintain n8n on your server.
- **Managed Environment**: n8n handles all backend infrastructure, so you can focus on building workflows.
- **Collaborative Workflows**: Share workflows with your team for seamless collaboration.
- **Scalability**: Easily scale your workflows and automation as needed.

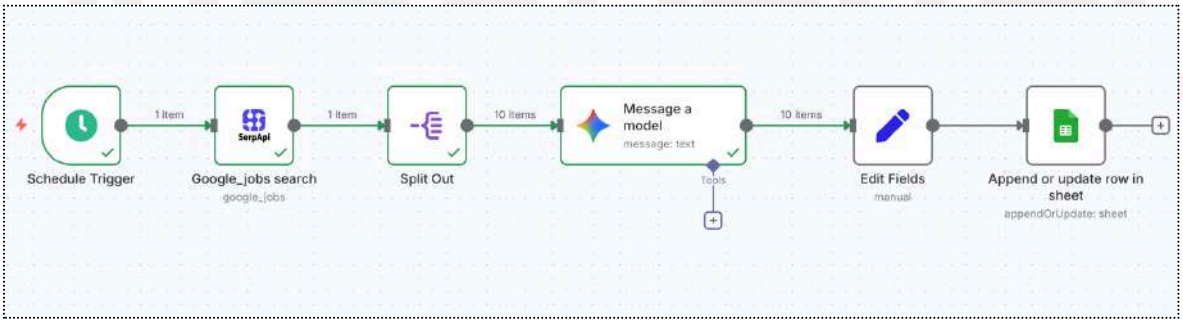
template:
https://drive.google.com/file/d/1fEJDKrJjSKSWNOIVMrG7MsnDGe6Pwaj_/view?usp=sharing

Project: Automated Job Search and Cover Letter Generator for Data Scientist Roles

This project automates the job search and cover letter generation process for **Data Scientist roles in Bangalore**. It searches for **relevant job listings**, generates **personalized cover letters** based on the **candidate's qualifications**, and assesses the **job match score** to evaluate how well the candidate fits the role, ultimately saving time and streamlining the application process.

Automated Job Search and Cover Letter Generation Workflow

1. **Scheduled Trigger**:
   - The process is triggered automatically every day at **10:00 AM**.
2. **Google Jobs Search**:
   - The system searches for **Data Scientist** job listings in **Bangalore** using **SerpApi**.
3. **Split Job Listings**:
   - The results are split into **individual job listings** to process each job separately.
4. **Generate Cover Letter**:
   - The **Google Gemini AI** generates a **personalized cover letter** for each job listing based on the candidate's resume details.
5. **Store Results in Google Sheets**:
   - The **job details** (title, company, location, description, etc.) and the **generated cover letter** are stored in **Google Sheets** for easy access and future use.



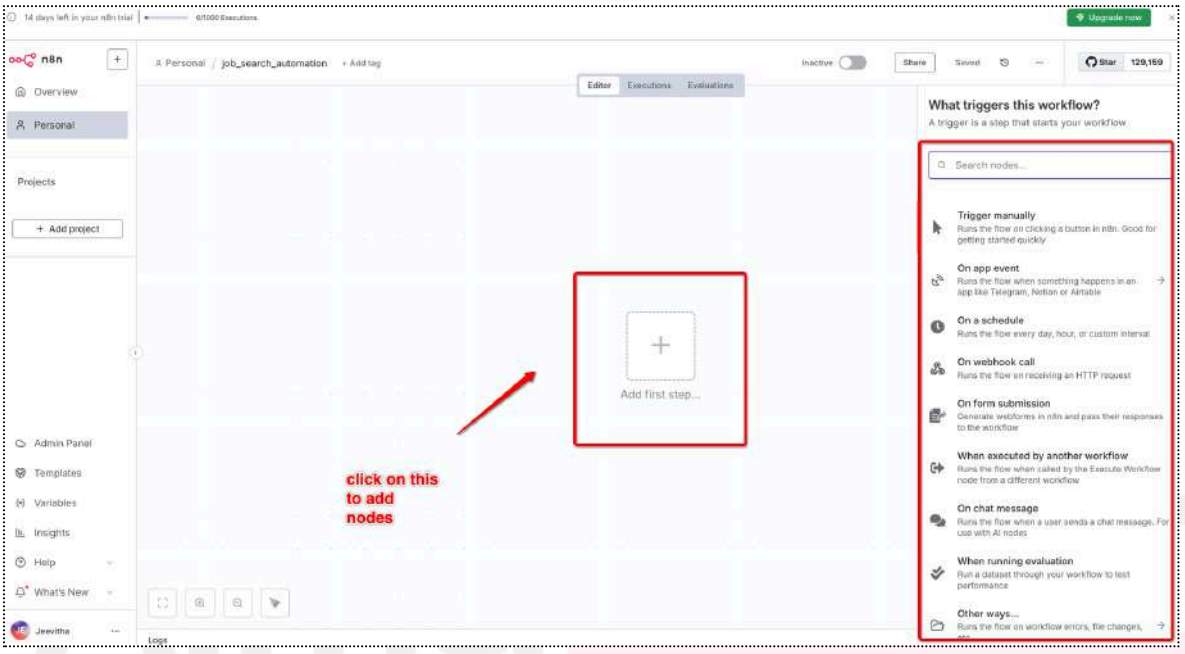
Step 1: How to *use* n8n Workflow

1.1 Open the n8n Workflow:

Navigate to the n8n dashboard and click on **create workflow**

1.2 rename the workflow to job_search_automation

1.3 click on "+" icon to add node and explore different nodes



Step 2: Add the Scheduled Trigger Node

Why Use the Scheduled Trigger Node?

The **Scheduled Trigger Node** is used to automate workflows by running them at specific times or intervals. Here's why it's important:

- **Automation**: It runs workflows automatically at scheduled times, removing the need for manual intervention.
- **Time-Based Execution**: You can set workflows to run at specific times, such as every day at 10:00 AM or weekly.
- **Efficiency**: It saves time by automating repetitive tasks like job searches or data processing.
- **Consistency**: Ensures tasks happen regularly without missing or delaying processes.

2.1 Click the "+" Icon

- Go to your **n8n dashboard** and open the workflow where you want to add the Scheduled Trigger node.

2.2 Search for "Scheduled Trigger

In the node search box that appears, type **"Scheduled Trigger"**.

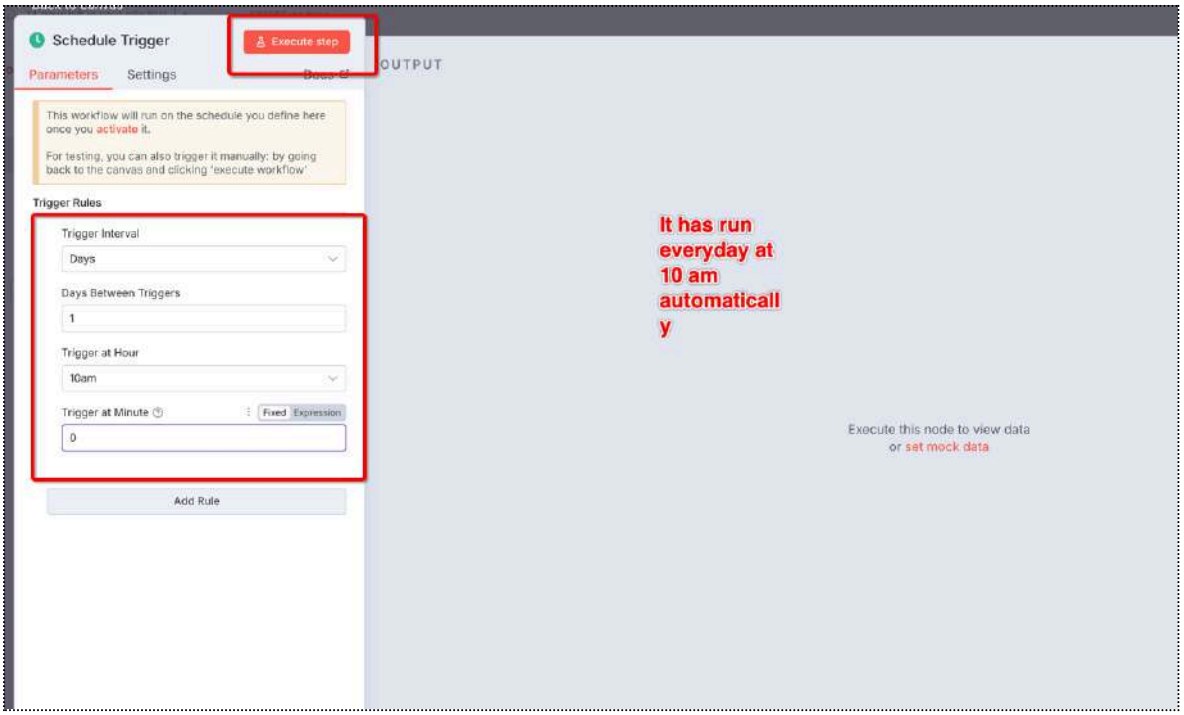
2.3 Search for "Scheduled Trigger"

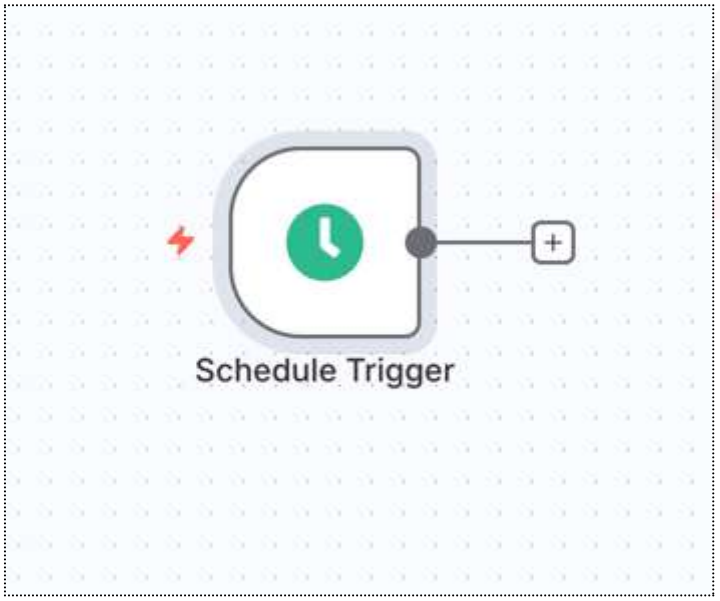
Select the **Scheduled Trigger node** from the search results.

2.4 Setting Up the Scheduled Trigger to Run Every Day at 10:00 AM

1. **Select the Scheduled Trigger Node** in your workflow.
2. **Configure Trigger Settings**:
a. **Trigger Interval**: Set to **Days**.
b. **Days Between Triggers**: Set to **1** (for every day).
c. **Trigger at Hour**: Set to **10 AM**.
d. **Trigger at Minute**: Set to **0**.
3. **Test the Trigger**:
a. Click **"Execute Workflow"** on the canvas to test it manually before activating.

Once activated, the workflow will run automatically **every day at 10:00 AM**.





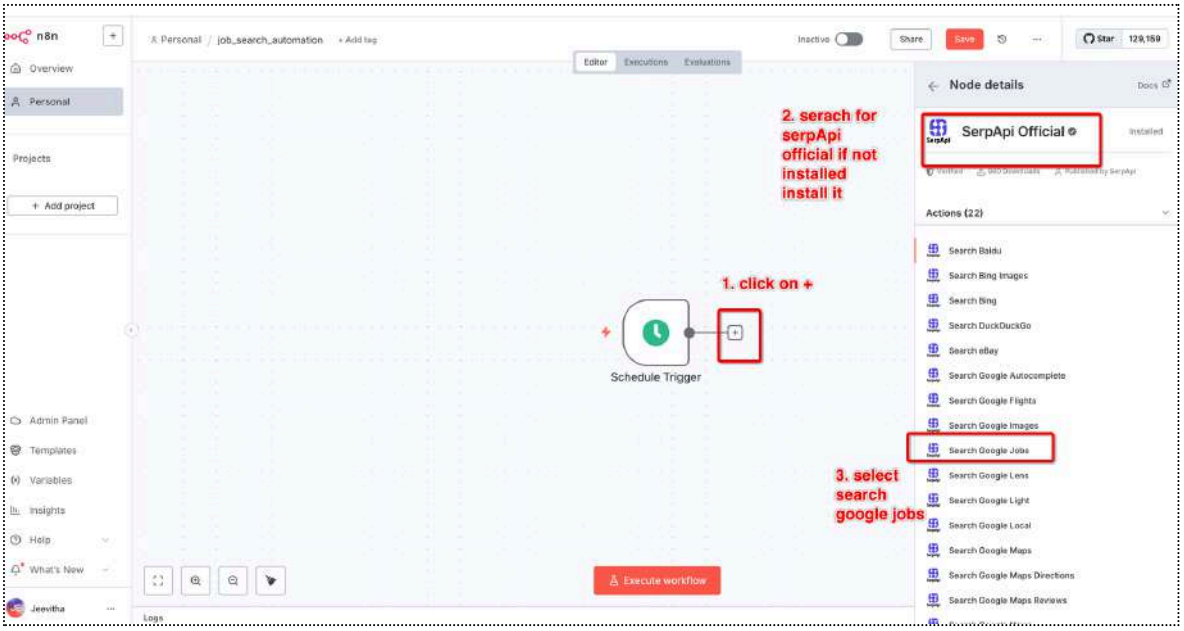
Step 3: Fetch Google Jobs Using SerpApi

This step is important because it automatically fetches relevant job listings for the specified role **(e.g., Data Scientist)** from **Google Jobs using SerpApi**. It saves time by eliminating the need for manual searches and ensures that only up-to-date, location-specific job listings (e.g., in Bangalore) are retrieved, making the job application process more efficient and automated.

3.1 Add the Google Jobs Search Node

- After the **Scheduled Trigger** node, click the **"+" icon** to add a new node.

- Search for **"SerpApi official and install it"** and then select the **Search Google jobs** node.
- Drag it to the canvas.

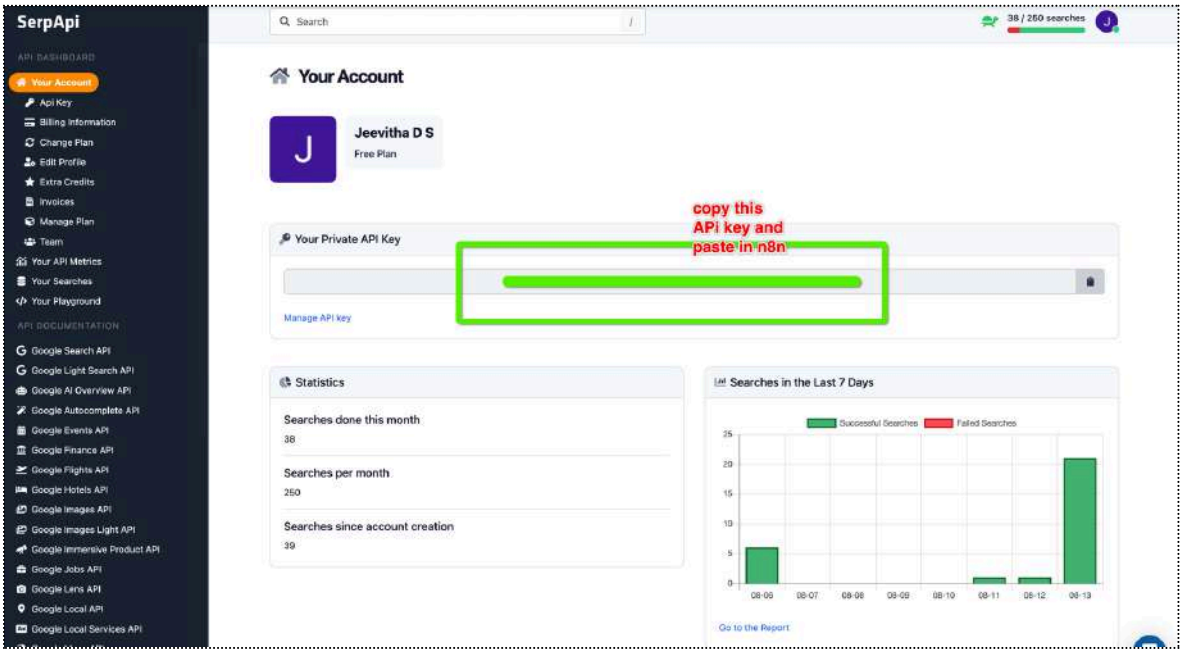


3.2 Configure the Google Jobs Search Node

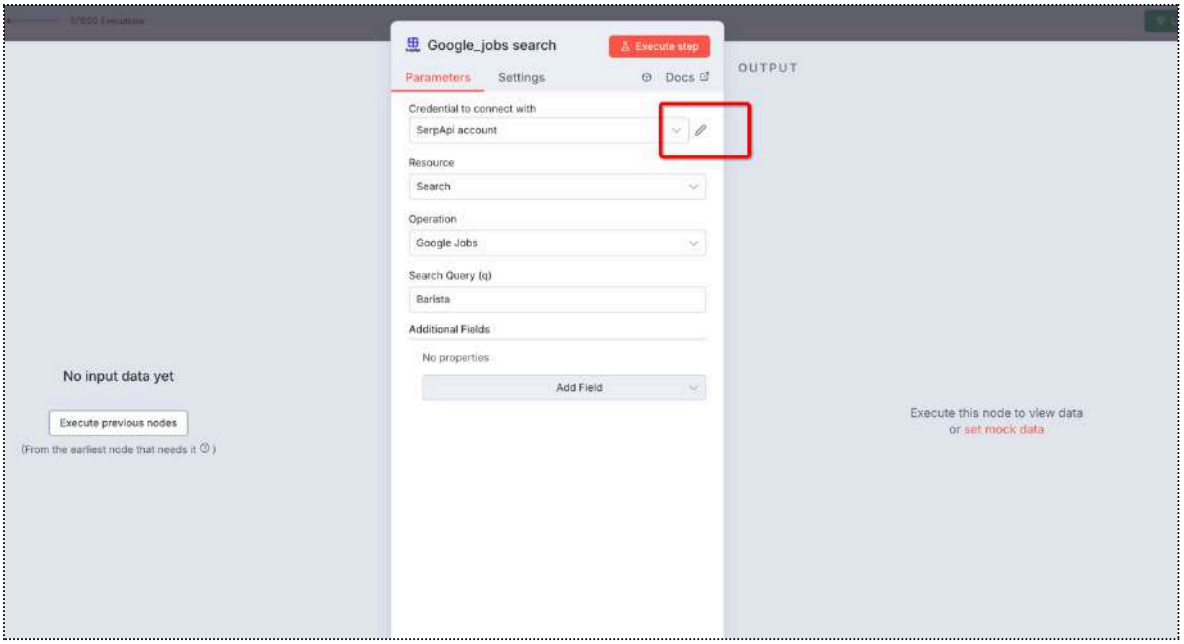
Credential to Connect with SerpApi Account:

- **Credentials**: Make sure you've connected your **SerpApi account** in the **Credentials tab** of the **Google Jobs Search node**.
- You can either **create new credentials** or **select an existing SerpApi account**.

Sign in using this https://serpapi.com/ and get API key and paste in n8n



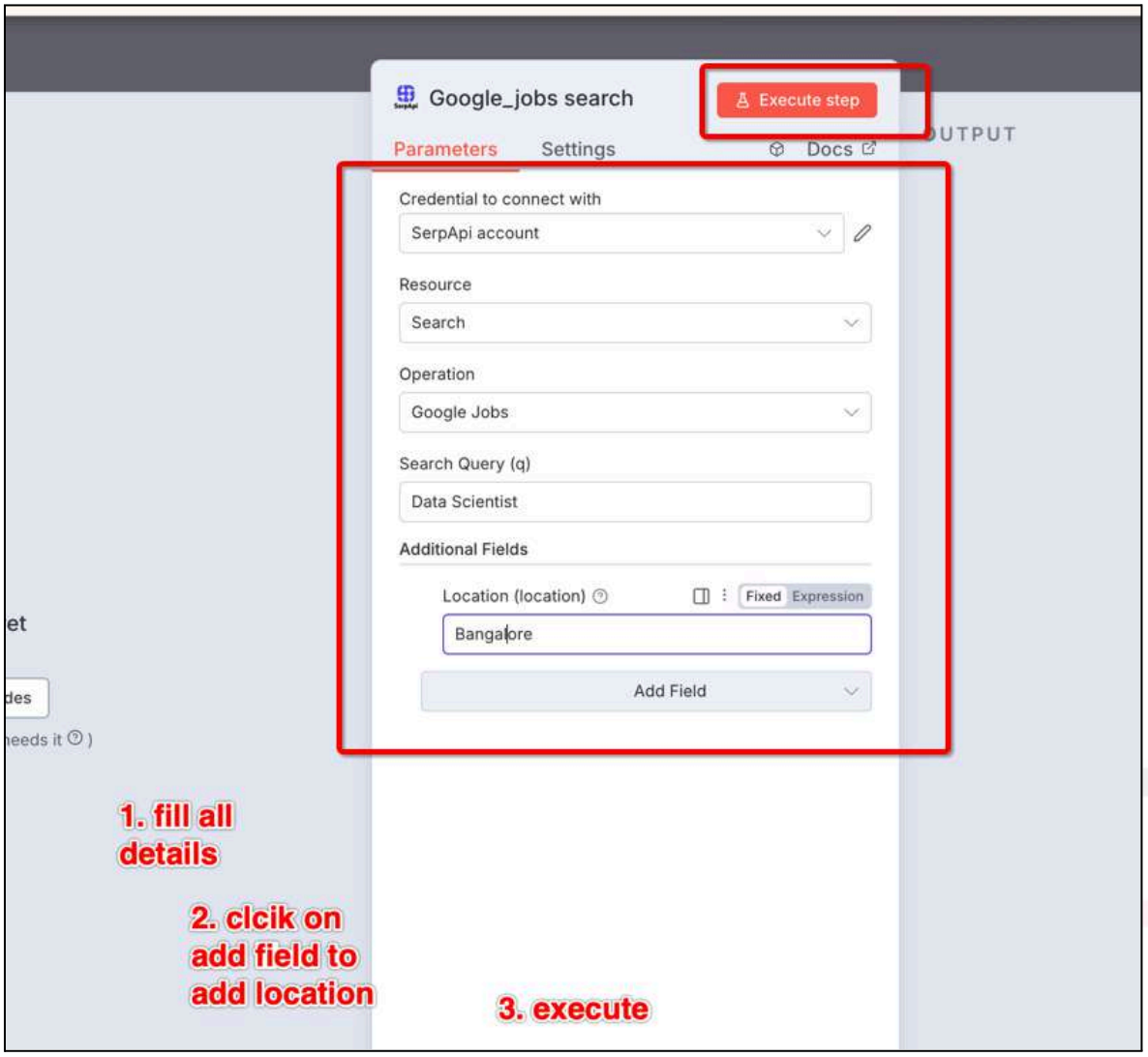
click on edit



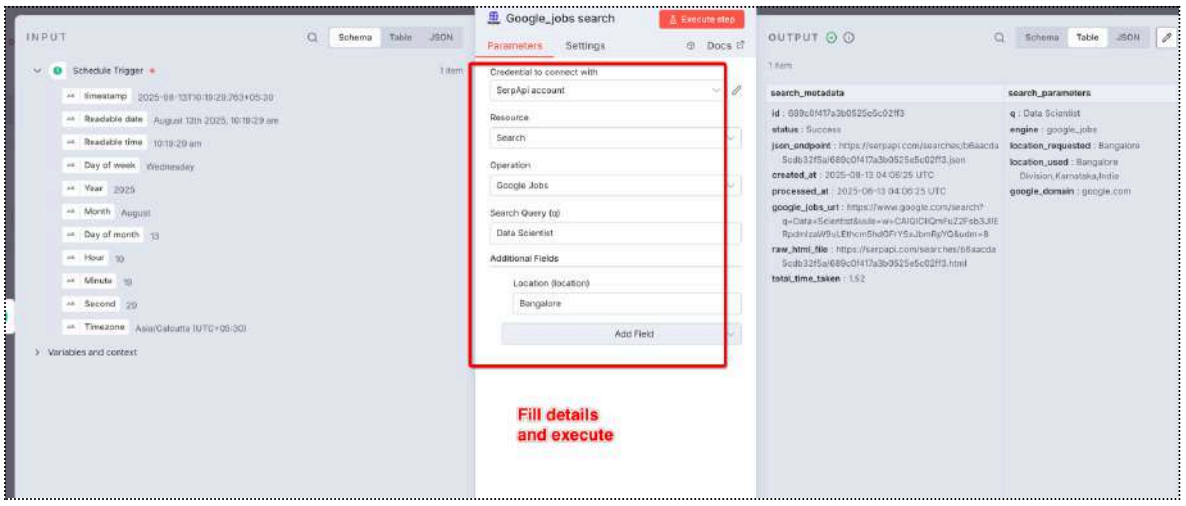
add serpapi credentials

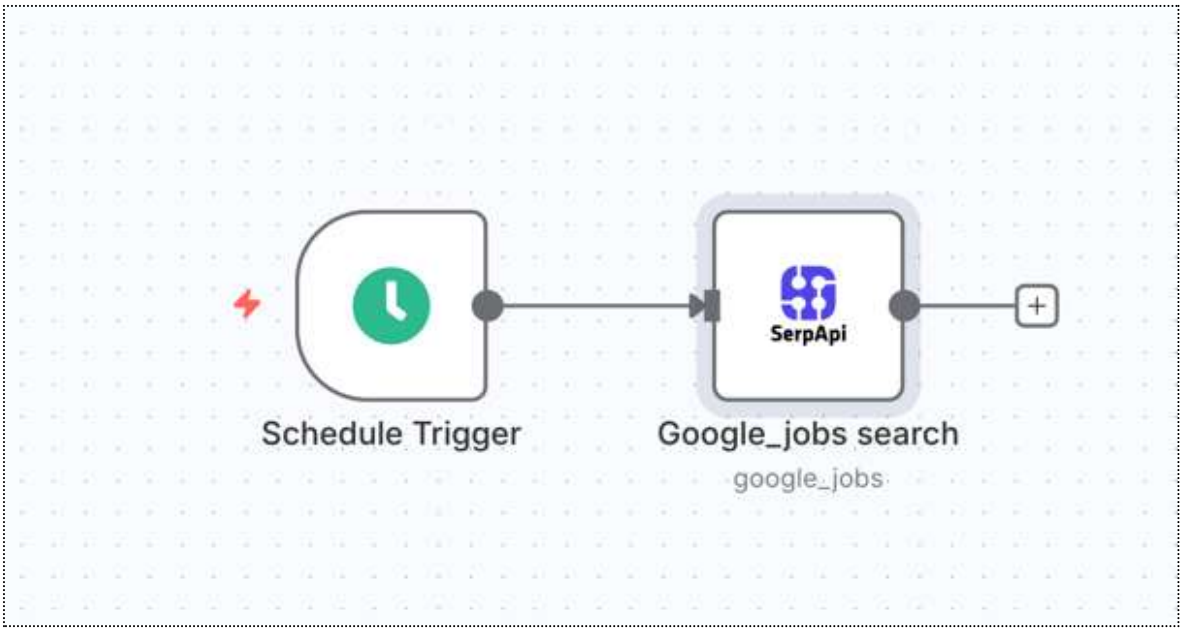
Fill all the details and execute the workflow

1. **Resource:**
- Set this to **Search** to initiate a job search query.
2. **Operation:**
- Set the operation to **Google Jobs** to search for job listings on Google Jobs.
3. **Search Query (q):**
- In the **Search Query (q)** field, type **"Data Scientist"** to search for Data Scientist job listings.
4. **Additional Fields:**
- **Location (location)**: Set the location field to **"Bangalore"** to search for jobs specifically in Bangalore.
5. **Add Field:**
- If you want to add more filters or details to your search, click on **Add Field** and configure it accordingly.



After execution it will fetch 10 job details at a time





Step 4:Steps to Configure the Split Node

Why Use the Split Node?

- **Use Case**: After fetching job listings from Google Jobs using **SerpApi**, the results come as a collection (a list of multiple job listings). To process each job individually (e.g., generating a personalized cover letter for each job), you need to **split the job listings** into individual items.
- The **Split Node** helps you break down a list of job results into separate entries, allowing each job listing to be handled independently in the next steps of the workflow.

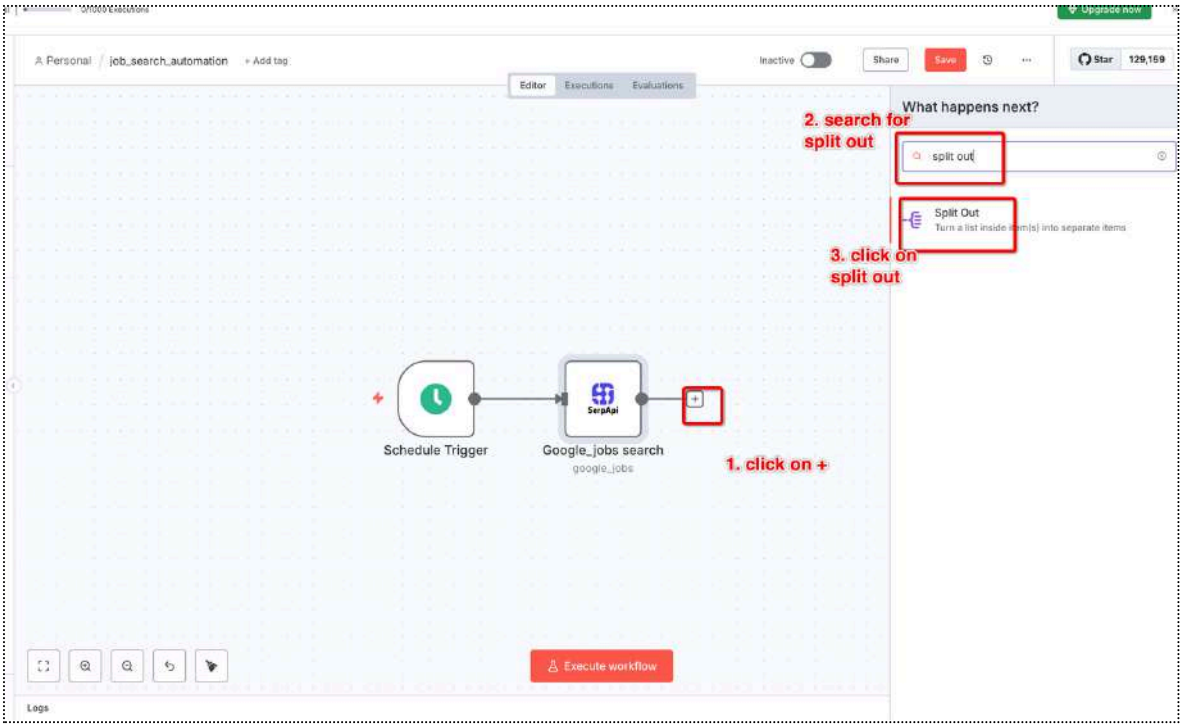
How the Split Node Works:

1. **Input**: The **Google Jobs Search node** returns multiple job listings.
2. **Output**: The **Split Node** processes each job listing separately, so that subsequent nodes (like generating cover letters) can work on each job individually.

4.1:Add the Split Node:

- After the **Google Jobs Search node**, click the **"+" icon** to add a new node.

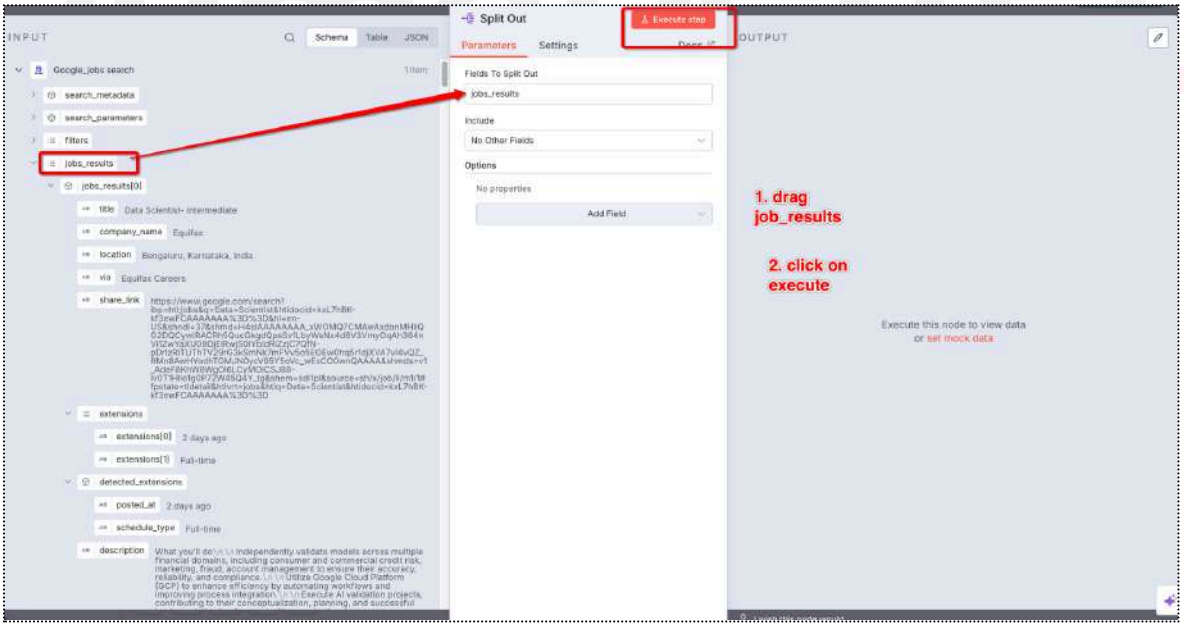
a. Search for "**Split Out**" and select the **Split Out node**.
b. Drag it to the canvas.

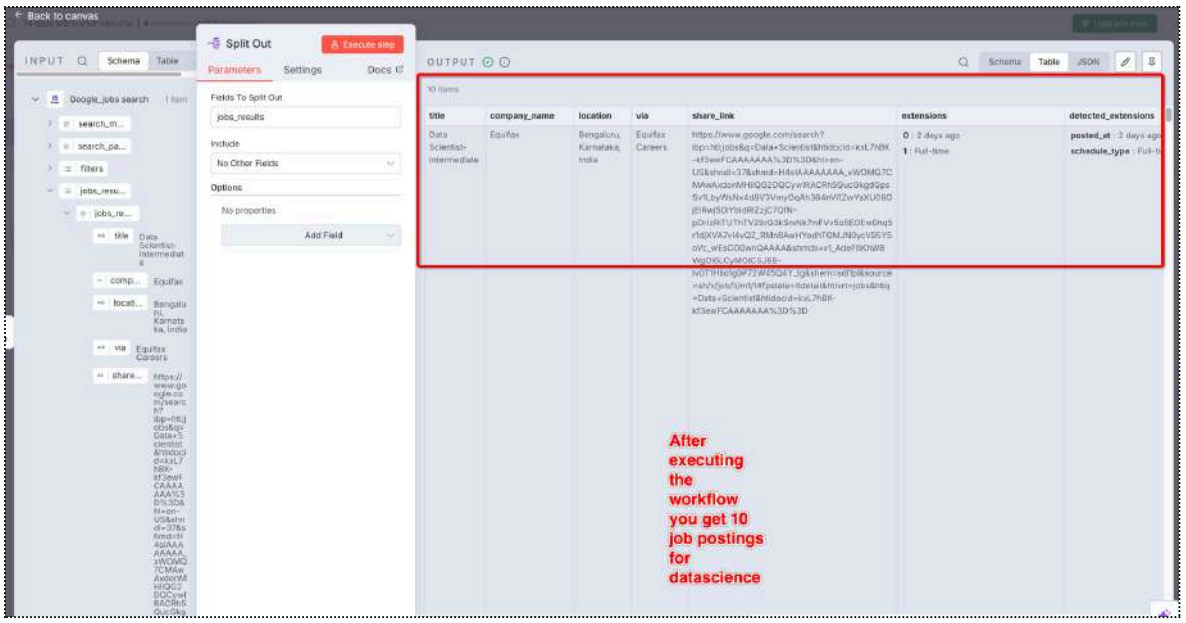


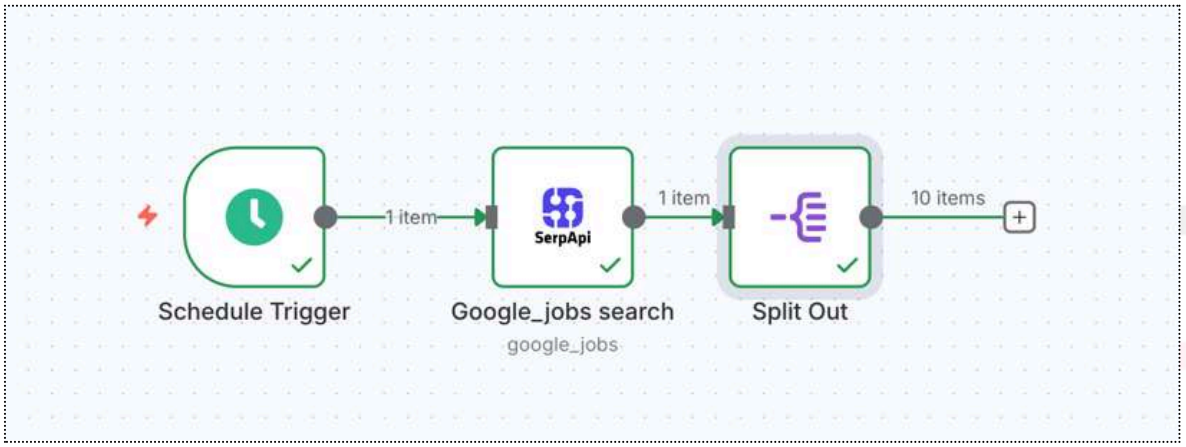
4.2:Configure the Split Node:

- In the **Split Out node settings**, set the **field to split out** as **"jobs_results"**.

a. This ensures that each job listing is processed separately.





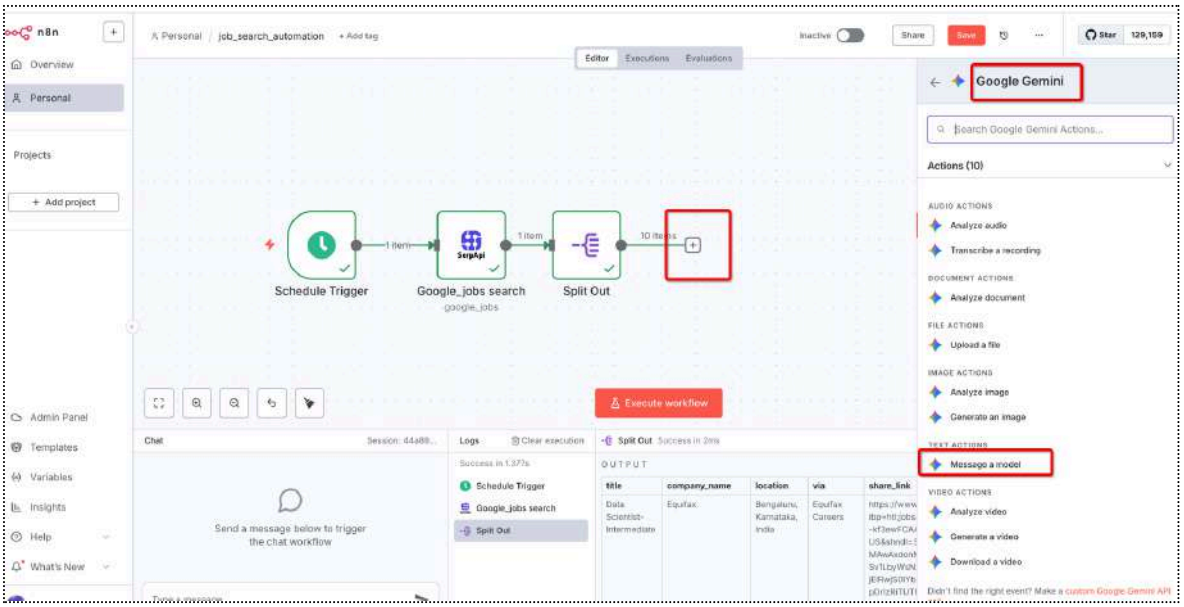


Step 5:Generate Personalized Cover Letter Using Google Gemini

In this step, we use the Google Gemini (PaLM) model to generate a personalized cover letter for each job listing based on the candidate's resume and job description. The model also assesses the job match score to evaluate how well the candidate's skills align with the job description.

5.1:Add the Google Gemini node

- click on +
- search for google gemini and click on it
- select message a model



5.2: Configure the Google Gemini Node:

1. **Credential to Connect with Google Gemini (PaLM) API**:

a. Ensure you have connected your **Google Gemini (PaLM) API credentials** in the **Credentials tab** of the node.

b. use this https://aistudio.google.com/app/apikey get the API key from gemini

2. **Resource**:

a. Set this to **Text** to work with text generation.

3. **Operation**:

a. Choose **"Message a Model"** to send a message to the **Gemini model**.

4. **Model**:

a. Select **"models/gemini-2.5-flash"** from the list of available models.

5. **Messages Configuration**:

a. **System Role**:

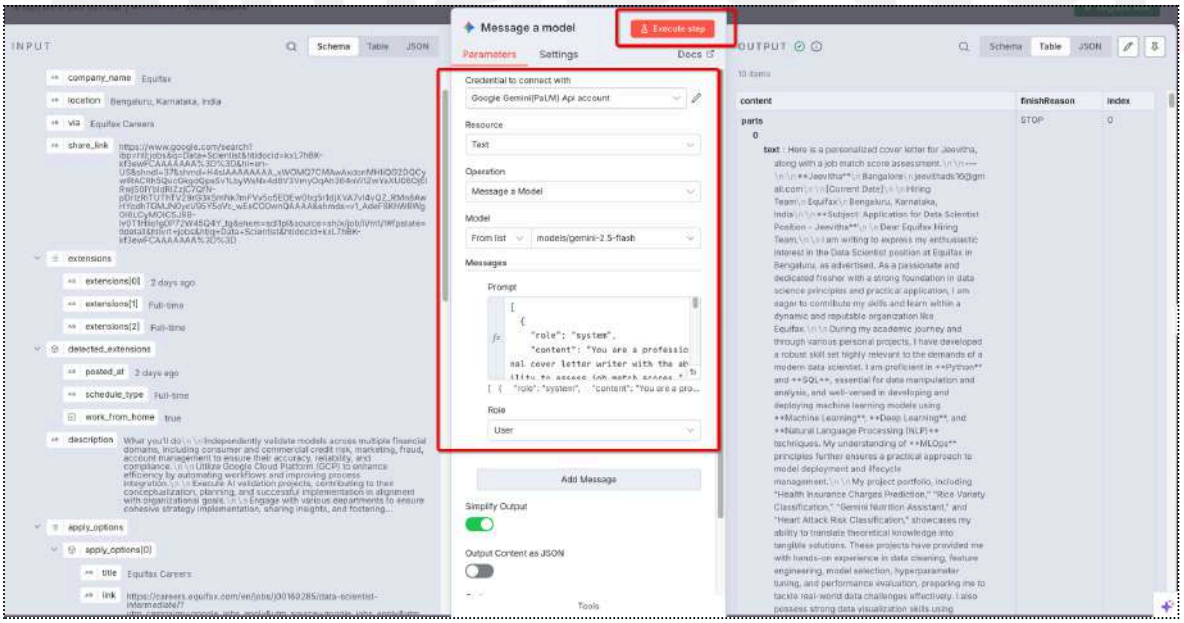
```
None

{
  "role": "system",
  "content": "You are a professional cover letter writer with the ability to assess job match scores."
}
```

b.

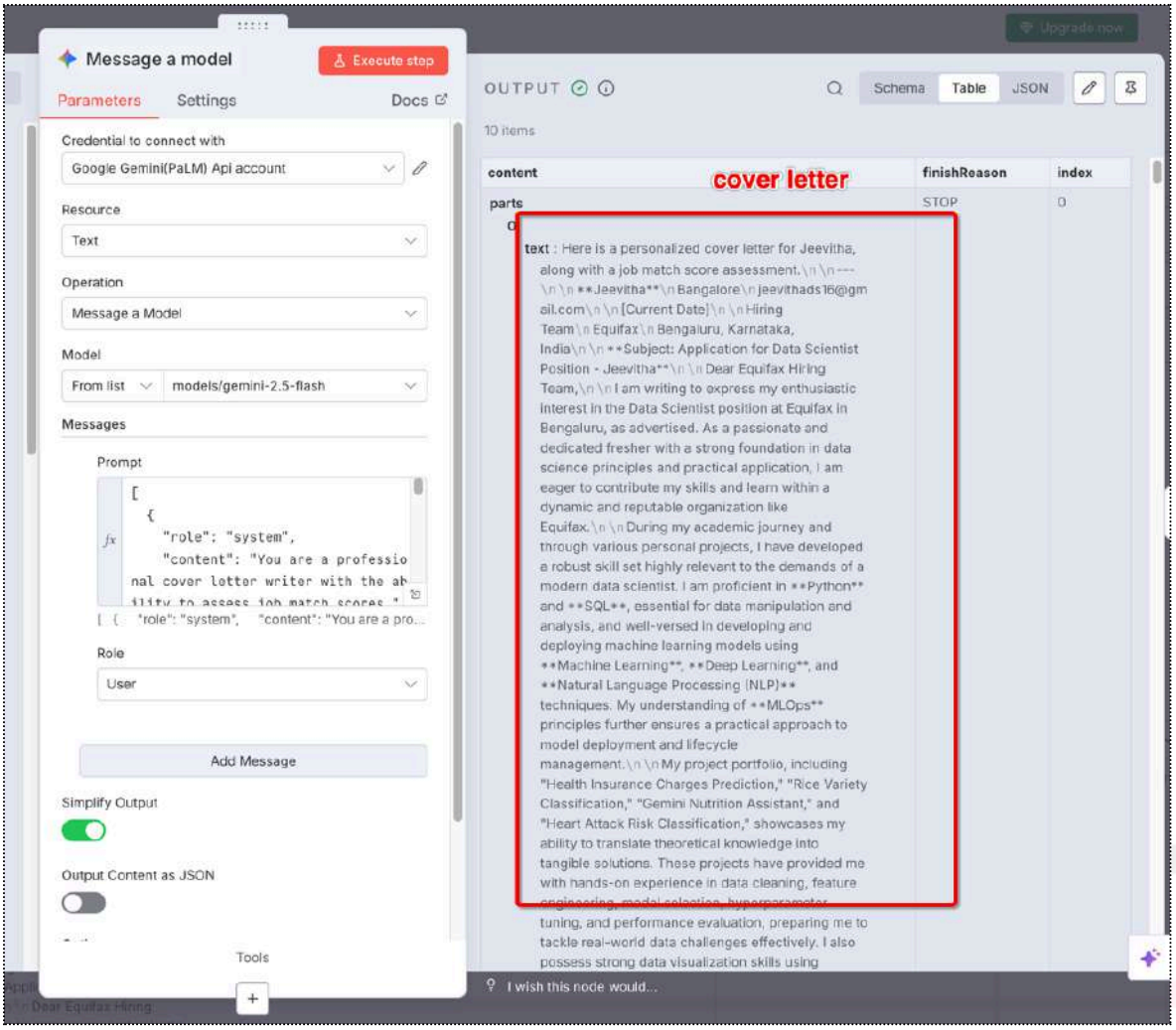
User Role:

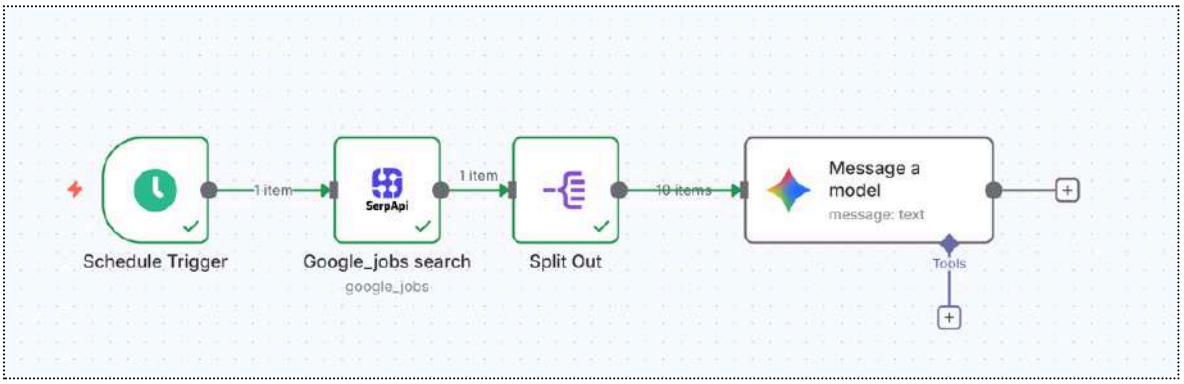
None

```
{
  "role": "user",
  "content": "Generate a personalized cover letter for a fresher Data Scientist position at {{ $json.company_name }} in Bangalore based on the following details:\n\nJob Title: Data Scientist\nCompany: {{ $json.company_name }}\nLocation: {{ $json.location }}\n\nResume Details:\n- Name: Jeevitha\n- Address: Bangalore\n- Email: jeevithads16@gmail.com\n- Skills: Python, SQL, PowerBI, Machine Learning, Excel, Tableau, Deep Learning, NLP, MLOps\n- Experience: {{ $json.experience }}\n- Education: {{ $json.education }}\n- Projects: Health Insurance Charges Prediction, Rice Variety Classification, Gemini Nutrition Assistant, Heart Attack Risk Classification\n\nThe candidate has a strong foundation in these skills and is eager to apply them in real-world data science challenges. The cover letter should reflect this and demonstrate enthusiasm for contributing to the company's success. Include the candidate's contact information at the top. Additionally, please assess the job match score based on the candidate's skills and the provided job description."
}
```



We got cover letter for every job


Message a model
Execute step
Parameters
Settings
Docs
Credential to connect with
Google Gemini(PaLM) Api account
Resource
Text
Operation
Message a Model
Model
From list
models/gemini-2.5-flash
Messages
Prompt
[
{
"role": "system",
"content": "You are a professio
nal cover letter writer with the ab
ility to assess job match scores."
}
]
Role
User
Add Message
Simplify Output
Output Content as JSON
Tools
OUTPUT
Schema
Table
JSON
10 items
content
cover letter
finishReason
index
parts
STOP
0
text : Here is a personalized cover letter for Jeevitha, along with a job match score assessment.\n\n---\n\n**Jeevitha**\nBangalore\njeevithads16@gmail.com\n\n[Current Date]\n\nHiring Team\nEquifax\nBengaluru, Karnataka, India\n\n**Subject: Application for Data Scientist Position - Jeevitha**\n\nDear Equifax Hiring Team,\n\nI am writing to express my enthusiastic interest in the Data Scientist position at Equifax in Bengaluru, as advertised. As a passionate and dedicated fresher with a strong foundation in data science principles and practical application, I am eager to contribute my skills and learn within a dynamic and reputable organization like Equifax.\n\nDuring my academic journey and through various personal projects, I have developed a robust skill set highly relevant to the demands of a modern data scientist. I am proficient in **Python** and **SQL**, essential for data manipulation and analysis, and well-versed in developing and deploying machine learning models using **Machine Learning**, **Deep Learning**, and **Natural Language Processing (NLP)** techniques. My understanding of **MLOps** principles further ensures a practical approach to model deployment and lifecycle management.\n\nMy project portfolio, including "Health Insurance Charges Prediction," "Rice Variety Classification," "Gemini Nutrition Assistant," and "Heart Attack Risk Classification," showcases my ability to translate theoretical knowledge into tangible solutions. These projects have provided me with hands-on experience in data cleaning, feature engineering, model selection, hyperparameter tuning, and performance evaluation, preparing me to tackle real-world data challenges effectively. I also possess strong data visualization skills using
I wish this node would...

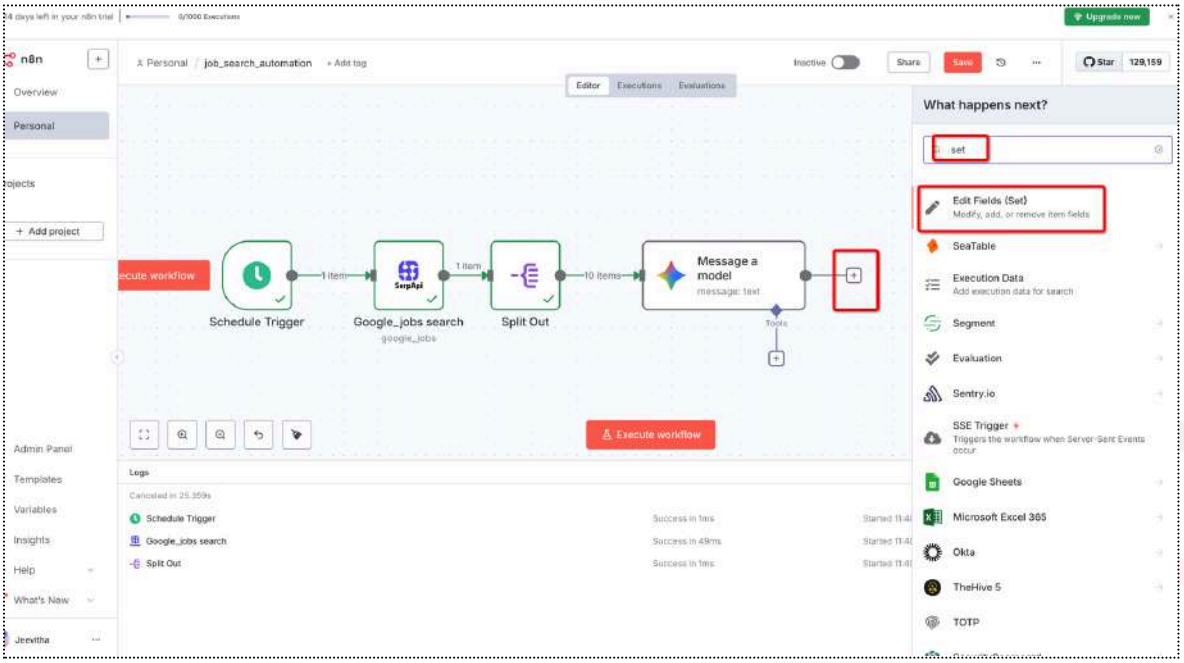

1 item
1 item
10 items
Schedule Trigger
Google_jobs search
google_jobs
Split Out
Message a model
message: text
Tools

Step 6: Configure the Set Node

- The **Set Node** allows you to store and organize the output from previous steps, such as the **cover letter** and **job details** generated by the **Google Gemini node**.
- It helps assign the generated data to specific fields for easy access in further steps (e.g., storing it in Google Sheets or sending it via email).

6.1 : Add the Set Node:

- After the **Google Gemini node**, click the **"+" icon** to add a new node.
- Search for "**Set**" and select the **Set node**.
- Drag it to the canvas.

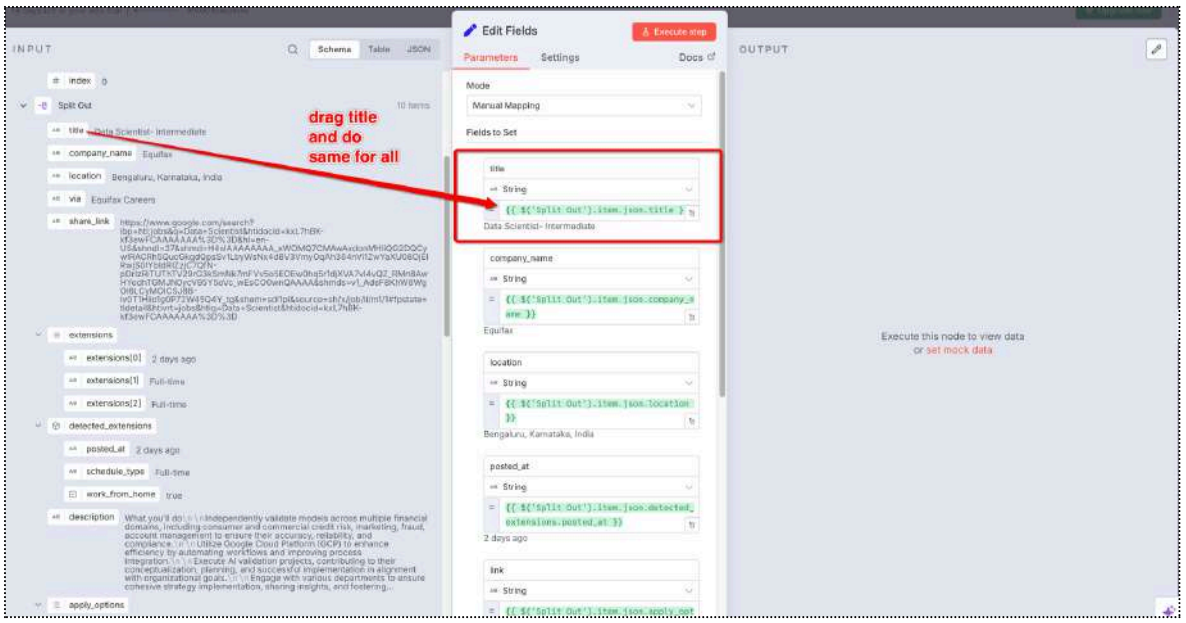


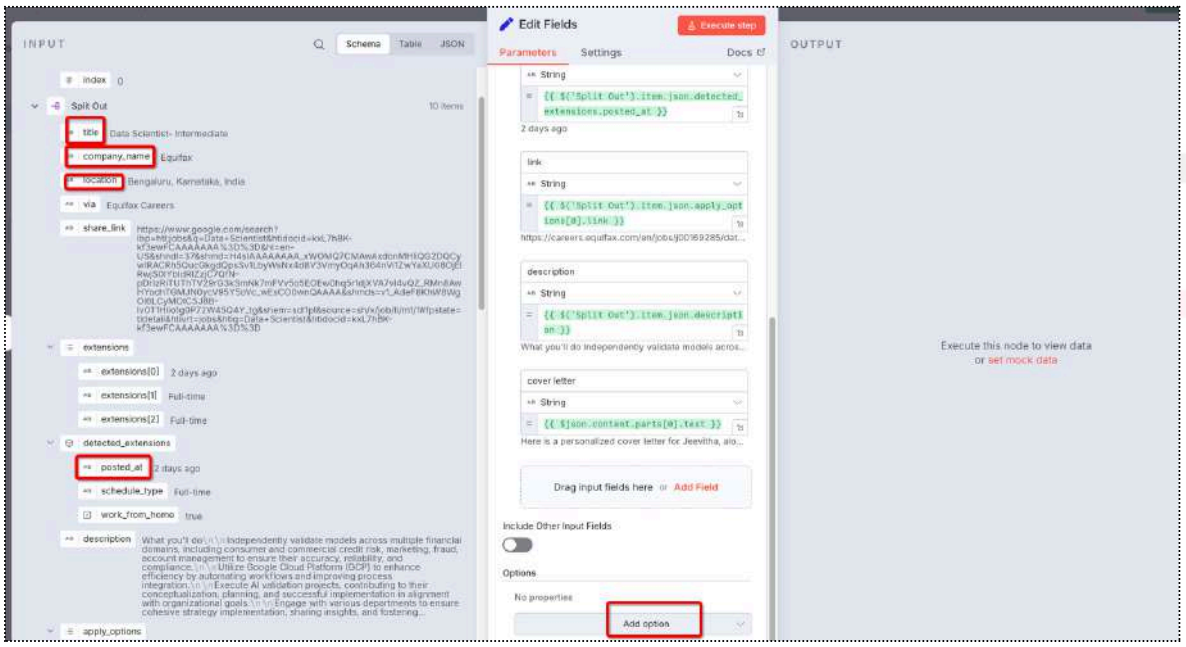
6.2 Manual Mapping:

In the Set Node settings, you can map the output fields from previous nodes to new fields in the Set node.

To do this:

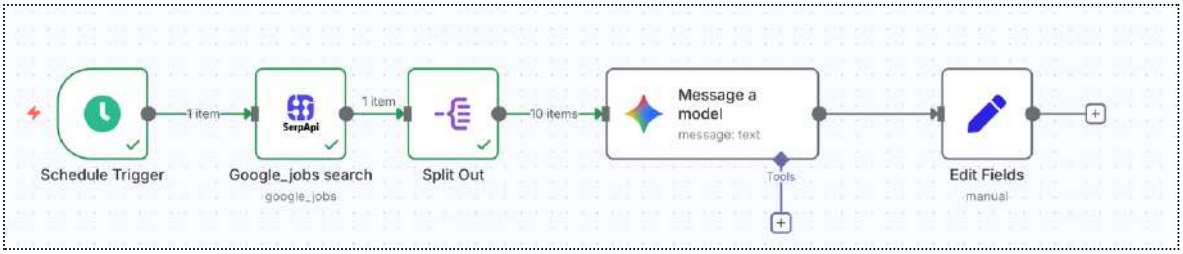
- Click on the "Add Field" button to create a new field (e.g., title, company_name, location, cover_letter, job_match_score).
- In the Value field, drag and drop the data from the previous nodes (e.g., Google Gemini, Split Out) into the corresponding field.





Simple Note on What to Drag in the Set Node

- **title**: {{ $('Split Out').item.json.title }}
- **company_name**: {{ $('Split Out').item.json.company_name }}
- **location**: {{ $('Split Out').item.json.location }}
- **posted_at**: {{ $('Split Out').item.json.detected_extensions.posted_at }}
- **link**: {{ $('Split Out').item.json.apply_options[0].link }}
- **description**: {{ $('Split Out').item.json.description }}
- **cover_letter**: {{ $json.content.parts[0].text }}



Step 7 : Store Results in Google Sheets

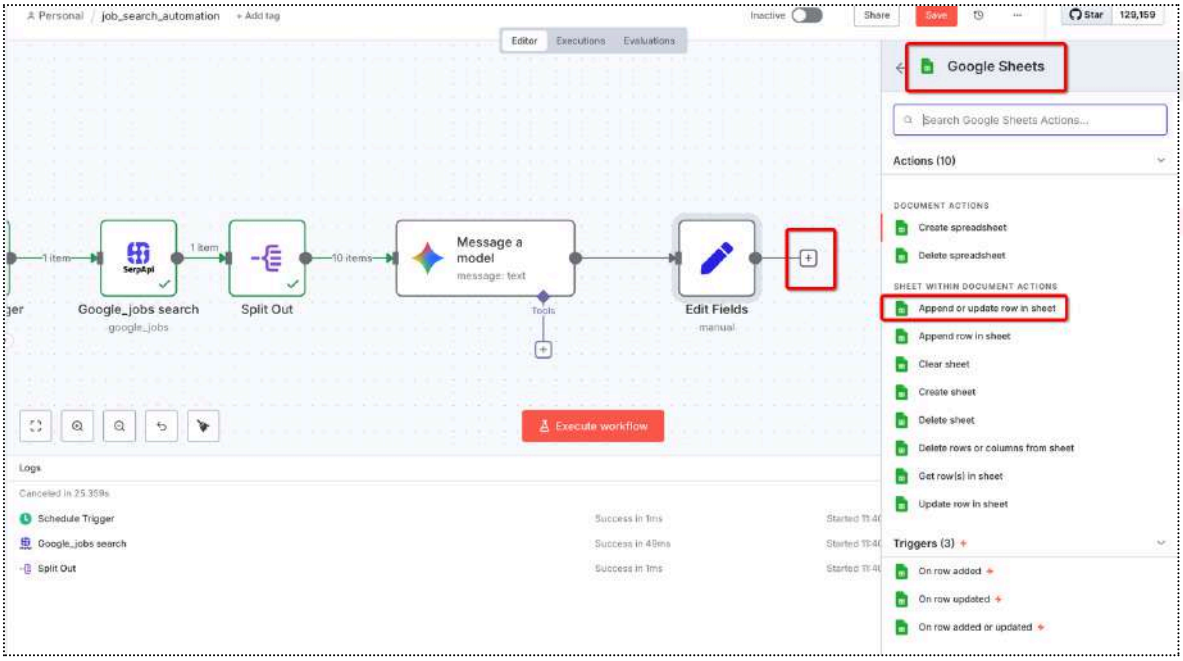
After configuring the Set Node, you can use the Google Sheets node to store the generated job listings and cover letters in a Google Sheet.

Why Use the Google Sheets Node?

The Google Sheets node allows you to save the processed job details (title, company, location, cover letter, etc.) directly to a Google Sheet for easy reference, tracking, or reporting.

7.1 Add the Google Sheets Node:

- After the Set Node, click the "+" icon to add a new node.
- Search for "Google Sheets" and select the **Append or update row in sheet**.
- Drag it to the canvas.



7.2 Steps to Connect Your Google Account to n8n

1. **Select Google Sheets API**:
a. When creating new credentials for Google Sheets, **select Google Sheets API** as the service to authenticate with.
2. **Click Continue**:
a. After selecting **Google Sheets API**, click **Continue** to proceed with the authentication process.
3. **Authenticate with Google**:

a. You will be redirected to the **Google sign-in page**.
b. **Sign in** with the **Google Account** you want to connect to n8n.
4. **Allow Permissions**:
a. Google will ask for your permission to allow n8n to access your Google Sheets and other services.
b. Click **Allow** to grant n8n the necessary permissions to access and modify your Google Sheets.
5. **Complete the Authentication**:
a. Once you grant access, you will be redirected back to n8n, and the connection will be established.
b. n8n will store the credentials, allowing the **Google Sheets node** to access your account.

7.3 Configure the Google Sheets Node

1. Credential to Connect with Google Sheets Account:

- Make sure you have connected your **Google Sheets account** in the **Credentials** tab of the node. This will allow the workflow to access your Google Sheets.

2. Resource:

- Set **Resource** to **Sheet Within Document** to interact with a specific sheet inside a document.

3. Operation:

- Choose "**Append or Update Row**". This allows new rows to be added to the sheet or updates to be made if the data already exists.

4. Document:

- **Document**: From the dropdown, select "**Youtube Data**" (or the name of your Google Sheets document).

5. Sheet:

- **Sheet**: Select "**Sheet1**" (or the sheet within your document where you want to store the data).

6. Mapping Column Mode:

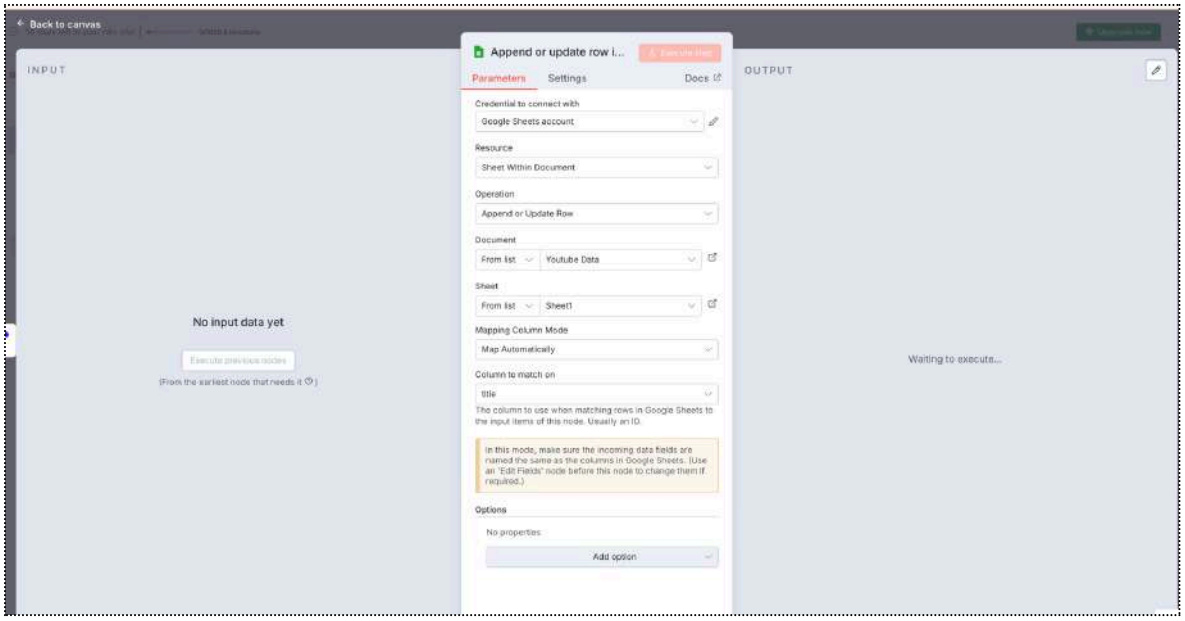
- Set **Mapping Column Mode** to **Map Automatically**. This ensures that the incoming data fields from previous nodes are automatically matched with the columns in your Google Sheet.

7. Column to Match On:

- **Column to match on**: Set this to "**title**" (or the column in Google Sheets that will be used to match the rows with the incoming data). Usually, an ID or a unique identifier is used for matching.

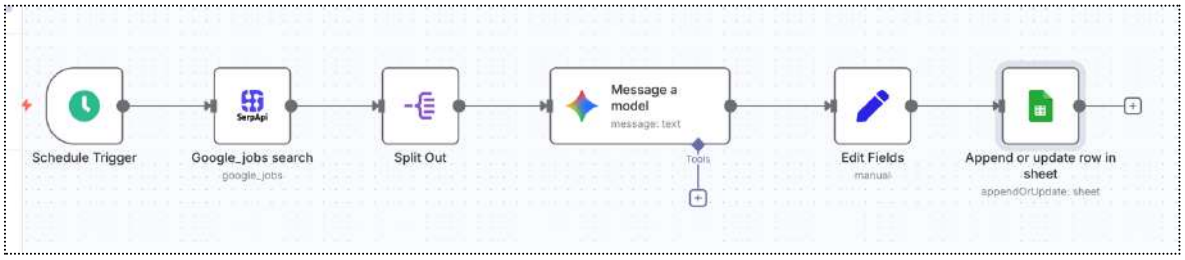
8. Options:

- Leave **Options** as **No properties** unless you need any additional configuration.



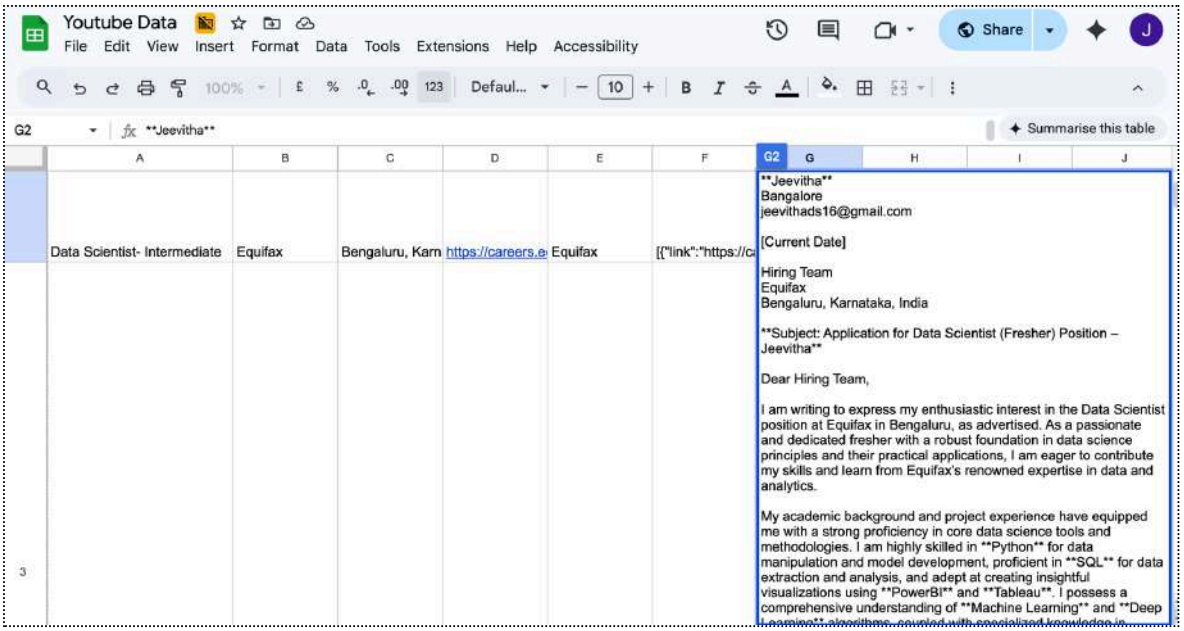
Example Configuration Summary:

- **Credential**: Google Sheets account
- **Resource**: Sheet Within Document
- **Operation**: Append or Update Row
- **Document**: Youtube Data
- **Sheet**: Sheet1
- **Mapping Column Mode**: Map Automatically
- **Column to match on**: title



7.4 Outcome:

Once configured, the **Google Sheets node** will append new job details (such as **job title**, **company name**, **location**, **cover letter**, etc.) to **Sheet1** in the **"Youtube Data"** Google Sheets document.



🎯 Conclusion

- An **Agent** is simply something that senses, decides, and acts.
- An **AI Agent** uses AI to make smarter decisions.
- **Agentic AI** is about agents that plan, reason, and collaborate like a team.

- **RAG** powers LLMs with real-world knowledge but is different from Agentic orchestration.
- **n8n** makes it easy to build these agentic workflows without coding.
- By combining n8n + AI, you can create powerful, real-world automations like **job search assistants, chatbots, and research pipelines**.

Top Hiring Partners Trusting our Alumni

Top **global companies trust our alumni** for their industry-ready expertise, hiring them confidently through our comprehensive training and personalized placement support.

Awards & Recognitions

AlmaBetter has won several accolades, including the Indian Achievers' Award for Emerging Edtech Company 2023, the Edutech Leadership Award by World Education Congress Awards 2023, and the Best Tech Start-up of the Year (Data Science) Award by TechIndia.


AlmaBetter
For a better tomorrow.